

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

КУРСОВАЯ РАБОТА
«Распределение Гомперца»

по дисциплине
«Теория вероятностей и математическая статистика»

Выполнили
студенты гр. 5131001/30003

Шевчук Н.Е.
Мальцев М.А.

Преподаватель

Изотова О.А.

Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

Введение	4
Цель.....	4
Задачи	4
1 Теоретическая справка	5
2 Вычисления статистик выборки X	7
3 Понятие эмпирической функции распределения.....	9
4 Понятие гистограммы.....	13
5 Описание параметров распределения	16
6 Понятие точечных оценок.....	19
6.1 Метод моментов	19
6.2 Метод максимального правдоподобия	20
6.3 Свойства оценок.....	21
6.3.1 Несмещенность	21
6.3.2 Состоятельность.....	22
6.3.3 Эффективность.....	24
6.3.4 R-эффективность.....	27
6.4 Графическое сравнение оценок параметров	27
7 Понятие интервальных оценок.....	28
7.1 Интервальная оценка параметра σ	29
7.2 Интервальная оценка параметра $\log$	29
8 Понятие статистических критериев	31
8.1 Гипотезы о параметрах распределения	31
8.2 Гипотезы о виде распределения	36
8.2.1 Критерий Колмогорова	36
8.2.2 Критерий χ^2	38
8.3 Гипотезы об однородности выборок	39
9 Заключение.....	44
Список источников.....	46
Приложение 1	47
Приложение 2	49
Приложение 3	50
Приложение 4	51
Приложение 5.....	53
Приложение 6	54

Приложение 7	56
Приложение 8	57
Приложение 9	59
Приложение 10	61
Приложение 11	64
Приложение 12	65
Приложение 13	68
Приложение 14	71
Приложение 15	73

ВВЕДЕНИЕ

Поиск закона распределения продолжительности жизни длительное время был горячо обсуждаемой проблемой для ученых. Вероятность смерти — это не самый удобный для анализа показатель. Прежде всего значения вероятности смерти зависят от величины возрастного интервала, для которого они рассчитаны. Пересчет значений вероятности смерти с одного возрастного интервала на другой с целью сопоставления данных должен проводиться в соответствии с алгеброй теории вероятностей, а не путем простого умножения или деления чисел. Таким образом, при расчетах с использованием вероятности смерти приходится постоянно контролировать соответствие выкладок алгебре теории вероятностей.

Одна из первых и наиболее удачных попыток математически выразить зависимость смертности от возраста была предпринята английским актуарием (специалистом по страхованию жизни) Б. Гомперцем еще в 1825 г. В данной курсовой работе мы более подробно исследуем это распределение.

ЦЕЛЬ

Исследовать распределение Гомперца с точки зрения математической статистики, написать программы на языке Python, вычисляющие различные математические статистики, оценки и статистические критерии распределения.

ЗАДАЧИ

1. Провести теоретическое исследование распределения Гомперца.
2. В среде jupyter notebook с помощью стандартных средств языка Python написать функции для вычисления статистик выборки X .
3. Исследовать эмпирическую функцию распределения, сравнить ее с теоретической.
4. Написать функцию для построения гистограммы на языке Python.
5. Исследовать параметры распределения.

6. Найти точные оценки параметров распределения.
7. Найти интервальные оценки параметров распределения.
8. Исследовать статистические критерии распределения

1 ТЕОРЕТИЧЕСКАЯ СПРАВКА

Распределение Гомперца было впервые предложено английским математиком и актуарием Бенджамином Гомперцем в 1825 году. Его работа была опубликована в журнале «Философские труды Лондонского королевского общества» под названием «О природе функции, выражающей закон человеческой смертности, и о новом способе определения стоимости страховых рисков». Гомперц изучал закономерности человеческой смертности и заметил, что вероятность смерти экспоненциально возрастает с возрастом после определённого периода (обычно после 25–30 лет). Его модель стала одной из первых попыток количественно описать процесс старения и смертности, что сделало её важной для актуарной науки и демографии. Изначально распределение разрабатывалось для расчёта стоимости пожизненных рент и страховых выплат, но позже нашло применение в других областях [1].

Гомперц предложил следующее теоретическое обоснование этой эмпирической закономерности: он предположил, что скорость уменьшения "сопротивляемости смерти" пропорциональна самой сопротивляемости. Поскольку интенсивность смертности μ служит мерой человеческой подверженности смерти, Гомперц принял в качестве меры сопротивляемости обратную ей величину $1/\mu(x)$, получив уравнение:

$$\frac{d}{dx} \frac{1}{\mu(x)} = -\alpha \frac{1}{\mu(x)},$$

где α – неотрицательный параметр. После интегрирования и упрощения получается формула:

$$\mu(x) = - \frac{dl(x)}{l(x)dx} = Re^{ax},$$

где $\mu(x)$ — интенсивность смертности в возрасте x , $l(x)$ — число доживающих до возраста x , а α и R — параметры уравнения. Эта формула, описывающая смертность людей старше 20 лет, была названа законом Гомперца, а ее параметры — параметрами Гомперца. Впоследствии этот закон стал широко использоваться для описания смертности лабораторных животных. [1], [2].

Модель Гомперца оказалась полезной для описания процессов, где наблюдается экспоненциальный рост с последующим замедлением или насыщением. В демографии оно используется для анализа и прогнозирования смертности по возрастным группам, что помогает строить таблицы смертности и оценивать продолжительность жизни. В биологии и медицине модель применяется для описания роста опухолей, где скорость роста замедляется по мере увеличения их размера, а также для изучения старения организмов. В экологии распределение помогает моделировать динамику популяций животных или растений с учетом ограничений среды. Современные исследования также применяют его в геронтологии для анализа поздних стадий жизни, где смертность начинает отклоняться от экспоненциального роста. Примером может служить работа Гаврилова и Гавриловой, где модель используется для изучения биологии продолжительности жизни, или исследования Миссова и Ленарта о смертности в старости. [2], [3].

Распределение Гомперца может быть использовано в задачах информационной безопасности для моделирования времени до успешной атаки на систему, где интенсивность атак экспоненциально возрастает со временем, например, при накоплении злоумышленником информации о системе.

2 ВЫЧИСЛЕНИЯ СТАТИСТИК ВЫБОРКИ X

Для работы над курсовой работой были установлены Python 3.11.9 и среда интерактивной разработки Jupyter Notebook. Были реализованы функции вычисления основных статистик выборки X, результаты которых представлены ниже:

1. Сумма элементов выборки: 1934.4925699999997

$$X_{\Sigma} = \sum_{i=1}^n x_i$$

Вычислялась с помощью встроенной функции Python `sum()`.

2. Выборочное среднее: 6.4483085666666655

$$\bar{X} = \frac{1}{n} \sum_{i=0}^n x_i$$

Вычислялась делением функции `sum()`, возвращающей сумму элементов выборки, на функцию `len()`, возвращающую количество элементов выборки.

3. Медиана: 6.423582

При $n = 2k$ (четное число элементов):

$$Me = \frac{X_{(k)} + X_{(k+1)}}{2}$$

При $n = 2k + 1$ (нечетное число элементов):

$$Me = X_{(k+1)}$$

В нашей выборке 300 элементов, поэтому использовалась формула для четного числа элементов. Для извлечения элементов $X_{(k)}$ и $X_{(k+1)}$ выборка предварительно сортировалась функцией `sorted()`.

4. Мода: Моды нет

Выборочная мода – элемент вариационного ряда с максимальной частотой. Вычислялась с помощью создания массива частот, т.е. массива, элементами которого было количество появлений каждого из элементов выборки. В нашей выборке каждый элемент встретился один раз, поэтому мода отсутствует.

5. Размах выборки: 0.38099799999999995

$$W_n = X_{(n)} - X_{(1)}$$

Вычислялась с помощью использования функций $\min()$ и $\max()$, для извлечения минимального и максимального элементов выборки соответственно.

6. Смещённая дисперсия: 0.006892634025465562

Моменты – характеристики, которые описывают те или иные свойства распределений. Дисперсия является 2-м моментом.

$$s^2 = \frac{1}{n} \sum_{i=0}^n (x_i - \bar{X})^2$$

Вычислялась с помощью уже реализованной функции для вычисления выборочного среднего.

7. Несмещённая дисперсия: 0.006915686313176149

Центральная случайная величина – отклонение случайной величины X от ее математического ожидания. Для таких величин моменты называются центральными и являются несмещёнными.

$$S^2 = \frac{1}{n-1} \sum_{i=0}^n (x_i - \bar{X})^2$$

Вычислялась аналогично смещённой дисперсии, но деление суммы квадратов производилась на уменьшенное на один количество элементов выборки.

8. Начальный момент 3-го порядка: 268.25905085095667

$$\bar{X}^3 = \frac{1}{n} \sum_{i=0}^n x_i^3$$

Вычислялась аналогично выборочному среднему, но перед суммированием значения элементов выборки возводились в куб.

9. Центральный момент 3-го порядка: 0.0006365691266045246

$$\bar{X}^3 = \frac{1}{n} \sum_{i=0}^n (x_i - \bar{X})^3$$

Вычислялась аналогично начальному моменту 3-го порядка, но в куб возводились не значения элементов выборки, а их разность со средним выборочным.

С листингом реализованных функций можно ознакомиться в приложении 1.

3 ПОНЯТИЕ ЭМПИРИЧЕСКОЙ ФУНКЦИИ РАСПРЕДЕЛЕНИЯ

Эмпирическая функция распределения, построенная по выборке $X_1, X_2, \dots, X_n$ размера n - случайная функция F_n^* :

$$F_n^*(x) = \frac{1}{n} \sum_{i=1}^n I(X_i < x), \forall x \in \mathbb{R}$$

Эмпирическая функция (1) распределения строится по вариационному ряду (т.е. упорядоченной выборке) следующим образом:

$$F_n^*(x) = \begin{cases} 0, x \leq X_{(1)} \\ \frac{k}{n}, X_{(k)} < x \leq X_{(k+1)}, \\ 1, x > X_{(n)} \end{cases} \quad (1)$$

Программно был реализован следующий алгоритм для построения ЭФР:

1. Случайным образом извлечь из выборки подвыборки размером 10, 100 и 200.
2. Отсортировать значения элементов каждой из подвыборок по возрастанию.
3. Вычислить значения ЭФР по формуле (1), приведенной выше. Значения по оси Ox для графика – отсортированная подвыборка.
4. Для корректного отображения графика добавить последнюю точку справа от максимального значения (чтобы ЭФР достигла единицы после последнего значения).

В Jupiter notebook была реализована программа для построения графиков эмпирической функции распределения. С её листингом можно

ознакомиться в приложении 2. Графики для подвыборок размера 10, 100, 200 приведены ниже, на рисунках Рисунок 1, Рисунок 2 и Рисунок 3:



Рисунок 1 – График эмпирической функции распределения для подвыборки размера 10



Рисунок 2 - График эмпирической функции распределения для подвыборки размера 100



Рисунок 3 - График эмпирической функции распределения для подвыборки размера 200

Функция распределения Гомперца имеет следующий вид:

$$F(x) = 1 - e^{-\frac{a}{b}(e^{bx}-1)}, \text{ где } x \geq 0, a > 0, b > 0, \text{ где}$$

$a > 0$ – параметр формы (базовый уровень смертности);

$b > 0$ – параметр масштаба (скорость роста смертности).

Так как по условия варианта необходимо исследовать параметры формы и смещения, то зададим параметру масштаба b фиксированное значение равное 1. Также, переобозначим параметр формы a как c и внесем параметр смещения loc . Тогда будем исследовать формулу:

$$F(x; c, loc) = 1 - e^{-c(e^{(x-loc)}-1)}$$

Была реализована программа для построения графика теоретической функции с различными базовыми параметрами. Листинг кода представлен в приложении 3. С полученным графиком можно ознакомиться на рисунке Рисунок 4.

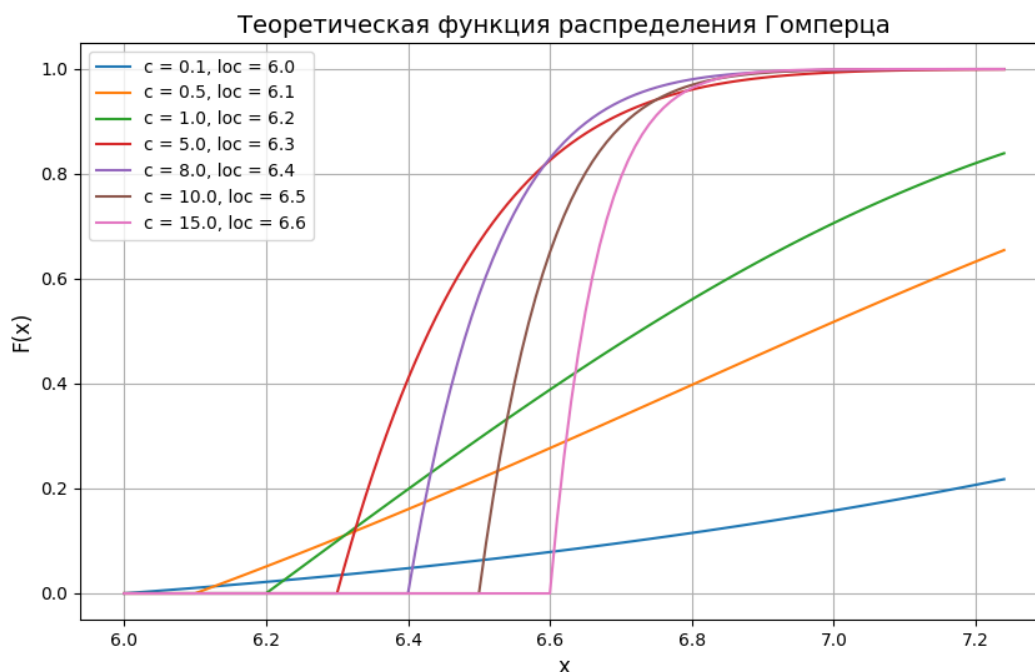


Рисунок 4 - График теоретической функции распределения Гомперца

Можно сделать вывод, что эмпирическая функция приближается к теоретической, только при больших размерах выборки. Также по форме

графика можно предположить, что параметр смещения loc будет равен 6,35, а параметр формы c будет числом, принадлежащим интервалу $[8.0, 15.0]$.

4 ПОНЯТИЕ ГИСТОГРАММЫ

Гистограмма — это графическое представление распределения данных (разновидность столбчатой диаграммы), которое показывает частоту (или относительную частоту) попадания значений выборки в заданные интервалы. Гистограмма состоит из прямоугольников, высота каждого из которых пропорциональна количеству (доле) элементов, попавших в соответствующий интервал. Площадь всех прямоугольников в сумме дает единицу.

Алгоритм построения гистограммы:

1. Определить диапазон значений выборки X ;
2. Разделить диапазон на k интервалов равной ширины:

$$h = \frac{x_{max} - x_{min}}{k}$$

Соответственно границы интервалов будут следующими:

$$[x_{min}, x_{min} + h), [x_{min} + h, x_{min} + 2h), \dots, [x_{min} + (k - 1)h, x_{max}]$$

3. Посчитать количество элементов в выборке, попавших в каждый интервал;
4. Нормировать частоты, чтобы получить плотность.

Количество интервалов определяется как корень из размера выборки.

Также был построен график (рисунок Рисунок 8) теоретической функции плотности распределения для сравнения полученных гистограмм с теоретической плотностью распределения Гомперца:

$$f(x; c, loc) = \begin{cases} \frac{d}{dx} F(x) = ce^{x-loc-c(e^{x-loc}-1)}, & x \geq loc \\ 0, & x < loc \end{cases}$$

Для построения гистограмм была реализована программа на Python в Jupyter Notebook, с листингом которой можно ознакомиться в приложении 4.

На рисунках Рисунок 5, Рисунок 6 и Рисунок 7, приведенных ниже, представлены полученные графики гистограмм.

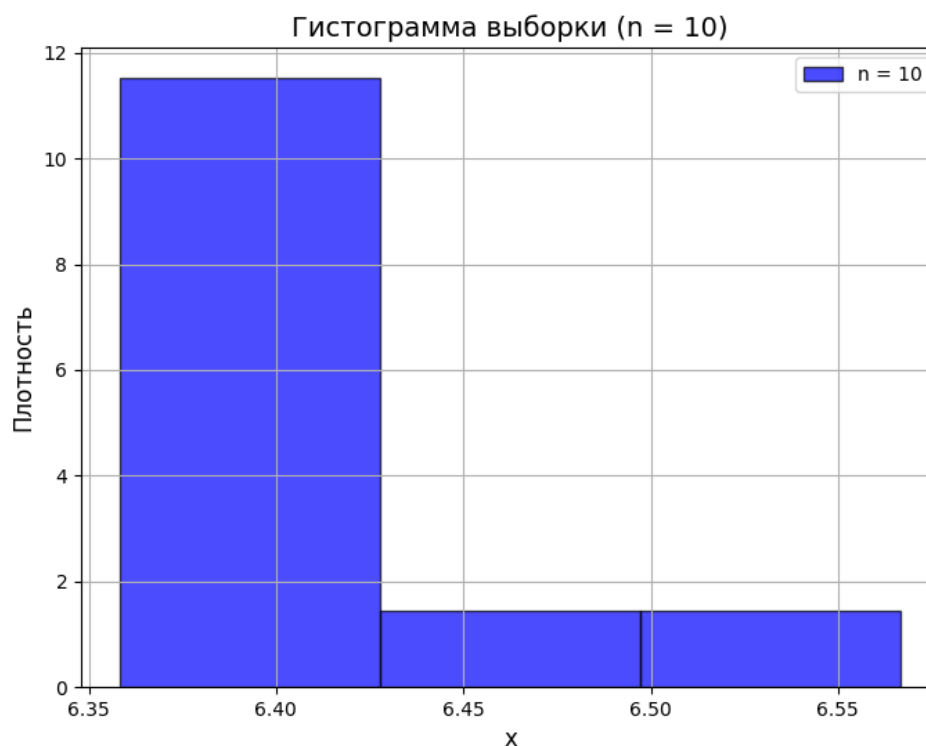


Рисунок 5 – График гистограммы для подвыборки размера 10

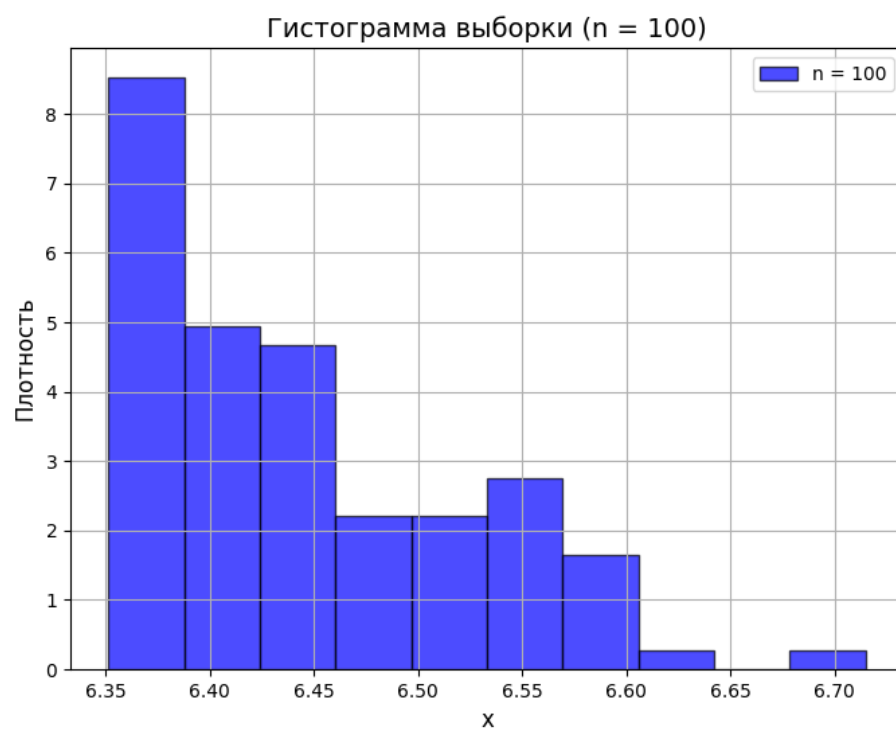


Рисунок 6 – График гистограммы для подвыборки размера 100

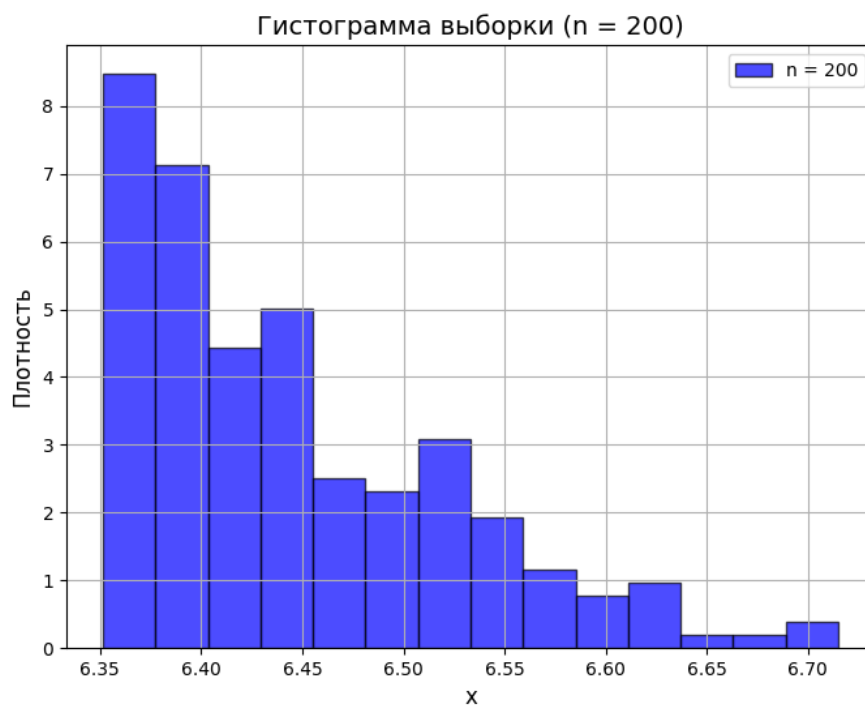


Рисунок 7 - График гистограммы для подвыборки размера 200

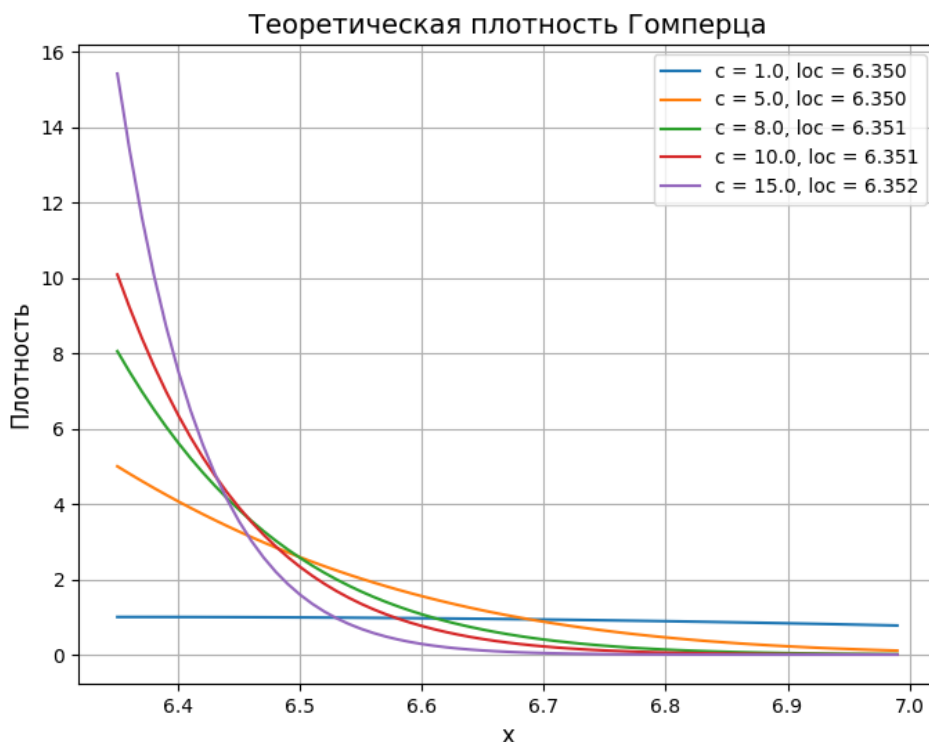


Рисунок 8 - График теоретической функции плотности распределения

Из анализа графиков можно сделать следующие выводы: по подвыборке размера 10 невозможно судить о плотности распределения. График гистограммы подвыборки размера 200 отдаленно напоминает теоретическую функцию плотности распределения Гомперца и позволяет уточнить оценку параметров из графиков ЭФР предыдущего пункта: параметр смещения loc равен 6.35, параметр формы c близок к числу 10.0.

5 ОПИСАНИЕ ПАРАМЕТРОВ РАСПРЕДЕЛЕНИЯ

Параметры распределения — величины, полностью описывающие какое-либо распределение и входящие в соответствующую функцию распределения в явном виде. Параметры по условию варианта: c - параметр формы, loc - параметр смещения.

Функция распределения Гомперца имеет следующий вид:

$$F(x; c, loc) = 1 - e^{-c(e^{(x-loc)} - 1)}$$

На рисунке **Рисунок 9** представлен график теоретической функции распределения Гомперца со случайно заданными параметрами $\log$ и c . Листинг кода для построения графика представлен в приложении 5.

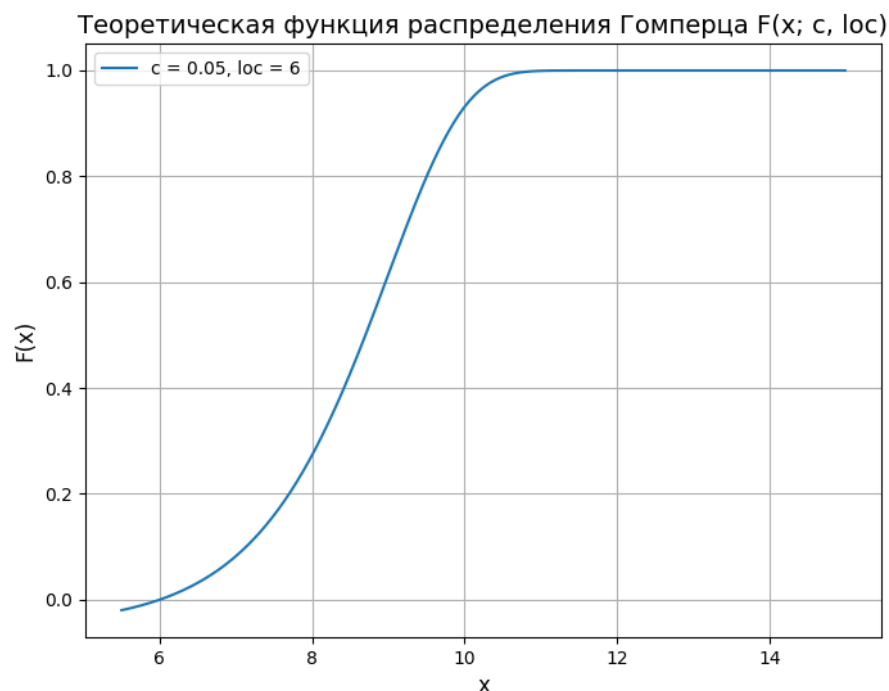


Рисунок 9 - График теоретической функции распределения Гомперца с фиксированными параметрами $c = 0,05$; $\log = 6,0$

Далее, чтобы исследовать влияние параметров на вид кривой, поочередно фиксировался один параметр, в то время как другому присваивалось два различных значения. С полученными графиками можно ознакомиться на рисунках **Рисунок 10** и **Рисунок 11**. Листинг кода, осуществляющего построение графиков, представлен в приложении 6.

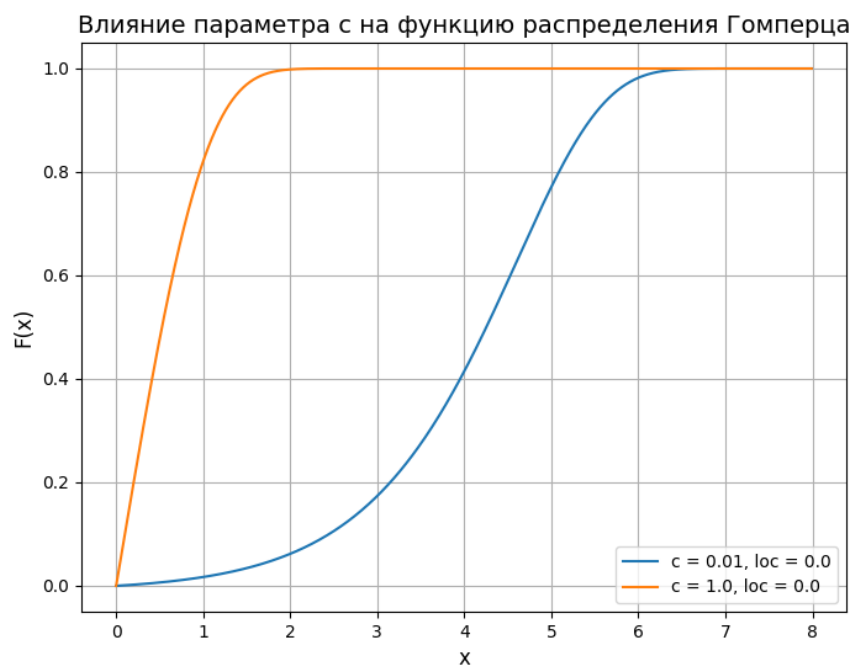


Рисунок 10 - График влияния параметра c на вид теоретической функции распределения

Из графика видно, что параметр c действительно отвечает за форму кривой. Он влияет на крутизну функции – насколько быстро значения возрастают и достигают единицы.

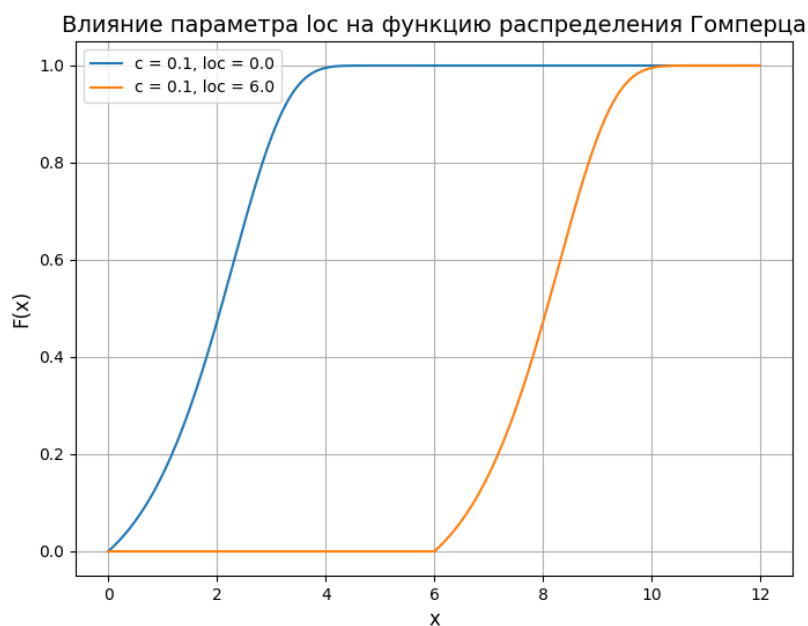


Рисунок 11 - График влияния параметра c на вид теоретической функции распределения

Из графика видно, что параметр loc влияет только на смещение графика по оси Ox .

6 ПОНЯТИЕ ТОЧЕЧНЫХ ОЦЕНОК

Пусть $X_1, \dots, X_n$ – случайная выборка для распределения зависящего от параметра $\theta \in \Theta$. Тогда статистику $\hat{\theta}(X_1, \dots, X_n)$, принимающую значения в Θ , называют точечной оценкой параметра θ .

Нам необходимо найти точечные оценки для параметров loc и c в заданной выборке, подчиняющей закону распределения Гомперца.

6.1 Метод моментов

Метод моментов состоит в том, что приравниваются соответствующие теоретические и эмпирические моменты и из полученных уравнений находятся оценки параметров. Составляется столько уравнений, сколько неизвестных параметров нужно оценить.

Функция плотности вероятности распределения Гомперца (с параметрами loc и c) имеет следующий вид:

$$f(x; c, loc) = \begin{cases} \frac{d}{dx} F(x) = ce^{x-loc-c(e^{x-loc}-1)}, & x \geq loc \\ 0, & x < loc \end{cases}$$

1. Сделаем замену $u = x - loc$, тогда $x = u + loc, u \geq 0$ и плотность:

$$f(u) = ce^{u-c(e^u-1)}$$

2. Вычислим математическое ожидание и приравняем его к среднему выборочному:

$$M[X] = M[U + loc] = M[U] + loc = \bar{X}$$

$$M[U] = \int_0^{+\infty} uce^{u-c(e^u-1)} du$$

Этот интеграл будем вычислять программно с помощью библиотек `scipy.integrate` и `scipy.optimize` Python.

Тогда оценку параметра loc можно будет вычислить, как:

$$\bar{X} - M[U] = loc$$

3. Дисперсия вычисляется как:

$$D[U] = M[U^2] - M[U]^2, \text{ где } M[U^2] = \int_0^{+\infty} u^2 c e^{u-c(e^u-1)} du$$

Этот интеграл также будем вычислять программно с помощью библиотек `scipy.integrate` и `scipy.optimize` Python. Значения интегралов вычисляются для различных c , а затем решается уравнение $M[U^2](c) - M[U]^2(c) = s^2$ (s^2 – вычисленная из выборки выборочная дисперсия). Таким образом осуществляется оценка параметра c , а затем производится оценка параметра loc (см. предыдущий пункт). Это корректно, так как $D[U] = D[X]$ (смещение не влияет на дисперсию, можно доказать формально).

Листинг программы, реализующей описанные действия представлена в приложении 8.

Полученные результаты:

$$\hat{c} = 10.221538, \widehat{loc} = 6.358577$$

6.2 Метод максимального правдоподобия

Метод максимального правдоподобия состоит в том, чтобы при конкретных значениях выборки $x_1, \dots, x_n$ найти такое значение θ , при котором функция $L(\theta)$ принимает максимальное значение.

Функция плотности вероятности распределения Гомперца (с параметрами loc и c) имеет следующий вид:

$$f(x; c, loc) = \begin{cases} \frac{d}{dx} F(x) = c e^{x-loc-c(e^{x-loc}-1)}, & x \geq loc \\ 0, & x < loc \end{cases}$$

1. Составим функцию правдоподобия:

$$L(c, loc) = \prod_{i=1}^n f(x_i; c, loc) = c^n e^{\sum_{i=1}^n (x_i - loc) - c \sum_{i=1}^n (e^{x_i - loc} - 1)}$$

2. Возьмем $\ln$ от обеих частей:

$$\ln L(c, loc) = n \ln c + \sum_{i=1}^n (x_i - loc) - c \sum_{i=1}^n (e^{x_i - loc} - 1)$$

3. Возьмем частные производные по c и loc и приравняем их к нулю:

$$3.1. \quad \frac{\partial \ln L}{\partial loc} = -n + c \sum_{i=1}^n e^{x_i - loc} = 0$$

$$c \sum_{i=1}^n e^{x_i - loc} = n$$

$$3.2. \quad \frac{\partial \ln L}{\partial c} = \frac{n}{c} - \sum_{i=1}^n (e^{x_i - loc} - 1) = 0$$

$$c = \frac{n}{\sum_{i=1}^n (e^{x_i - loc} - 1)}$$

При приравнивании уравнений 3.1. и 3.2. друг к другу получаем тривиальное решение $n = n$. Поэтому примем $\widehat{loc} = \min(X_i)$ и используя его вычислим $\hat{c}$. Для этого был реализован код, с листингом которого можно ознакомиться в приложении 7. Полученные значения:

$$\hat{c} = 9.453330, \widehat{loc} = 6.351309$$

6.3 Свойства оценок

6.3.1 Несмещенность

Оценка $\hat{\theta}$ называется несмещенной, если $M[\hat{\theta}] - \theta = 0$.

Чтобы найти $M[\hat{c}]$ и $M[\widehat{loc}]$ необходимо выразить $\hat{c}$ и $\widehat{loc}$ как функции выборки X и выразить математическое ожидание:

$$M[\hat{c}] = \int \hat{c}(X_1, \dots, X_n) f(X_1, \dots, X_n) dX_1 \dots dX_n$$

Где $f(X_1, \dots, X_n) = \prod_{i=1}^n f(x_i; c, loc)$

Полученные в ММП уравнение $c \sum_{i=1}^n e^{x_i - loc} = n$ не позволяет явно выразить $\hat{c}$, от которого также зависит $\widehat{loc}$. Выражение получится нелинейным.

При вычислении оценок параметров методом моментов использовалось уравнение:

$$\bar{X} - M[U](\hat{c}) = loc$$

Отсюда: $M[\widehat{loc}] = M[\bar{X}] - M[M[U](\hat{c})]$.

Рассмотрим элементы разности:

$$1. \quad M[\bar{X}] = M\left[\frac{1}{n} \sum_{i=1}^n x_i\right] = \frac{1}{n} \sum_{i=1}^n M[x_i] = M[x_i] = M[U] + loc$$

Где $M[U]$ – истинное математическое ожидание распределения Гомперца $F(x; c, 0)$.

2. $M[U](c)$ – теоретическое математическое ожидание U , зависящее от параметра c , для вычисления которого мы использовали второй момент:

$$M[U^2](\hat{c}) - M[U]^2(\hat{c}) = s^2$$

Т.е. c является случайной величиной (т.к. это оценка, полученная методом моментов).

$$M[U] = \int_0^{+\infty} u c e^{u-c(e^u-1)} du$$

Этот интеграл не имеет простого аналитического решения (вычислялся программно), кроме того $M[U](\hat{c})$ представляет собой нелинейную функцию. Следовательно $M[M[U](\hat{c})] \neq M[U]$.

В результате получаем:

$$M[\widehat{loc}] = M[\bar{X}] - M[M[U](\hat{c})] = M[U] + loc - M[M[U](\hat{c})] \neq loc$$

Следовательно, оценки обоих параметров обоими методами не являются несмещёнными.

6.3.2 Состоятельность

Оценка $\hat{\theta}$ называется состоятельной, если $\hat{\theta}_n \xrightarrow{P} \theta$ при $n \rightarrow \infty$.

1. Метод моментов:

Оценка параметра c методом моментов вычислялась из выражения:

$$M[U^2](\hat{c}) - M[U]^2(\hat{c}) = s^2$$

Где, $U = X - loc$, $s^2 = \frac{1}{n} \sum_{i=0}^n (x_i - \bar{X})^2$ – выборочная дисперсия, $M[U^2](\hat{c}), M[U]^2(\hat{c})$ – теоретические моменты распределения Гомперца $F(x; c, 0)$. Обозначим: $G(\hat{c}) = M[U^2](\hat{c}) - M[U]^2(\hat{c})$. Тогда уравнение второго момента для вычисления $\hat{c}$ будет выглядеть так: $G(\hat{c}) = s^2$.

s^2 - состоятельная оценка по закону больших чисел. : $G(\hat{c})$ непрерывна и монотонна (следует из производной $G'(\hat{c})$). По теореме о непрерывном отображении (непрерывные функции сохраняют пределы, даже если их

аргументами являются последовательности случайных величин): $s^2 \xrightarrow{P} G(\hat{c})$, G обратима, то $\hat{c} = G^{-1}(s^2) \xrightarrow{P} G^{-1}(G(\hat{c})) = c$. Следовательно $\hat{c}$, полученная методом моментов – состоятельная оценка.

Для вычисления оценки параметра loc использовалось следующее выражение:

$$\bar{X} - M[U](\hat{c}) = \widehat{loc}$$

$\bar{X}$ – состоятельная оценка по закону больших чисел, $\bar{X} \xrightarrow{P} M[X] = M[U] + loc$. Выше показана сходимость $\hat{c} \xrightarrow{P} c$, откуда следует $M[U](\hat{c}) \xrightarrow{P} M[U]$.

Итого:

$$\widehat{loc} = \bar{X} - M[U](\hat{c}) \xrightarrow{P} M[U] + loc - M[U] = loc.$$

Следовательно, оценка параметра loc , вычисленная методом моментов является состоятельной оценкой.

2. Метод максимального правдоподобия:

При работе с функцией правдоподобия было получено следующее выражение:

$$\hat{c} \sum_{i=1}^n (e^{x_i - \widehat{loc}} - 1) = n$$

По закону больших чисел:

$$\frac{1}{n} \sum_{i=1}^n (e^{x_i - \widehat{loc}} - 1) \xrightarrow{P} M[e^{X - \widehat{loc}} - 1]$$

$$\frac{1}{n} \sum_{i=1}^n e^{x_i - \widehat{loc}} \xrightarrow{P} M[e^{X - \widehat{loc}}]$$

$$X - \widehat{loc} = (X - loc) + (loc - \widehat{loc}) = U + (loc - \widehat{loc})$$

$$e^{X - \widehat{loc}} = e^{U + (loc - \widehat{loc})} = e^U e^{loc - \widehat{loc}}$$

$$M[e^{X - \widehat{loc}}] = M[e^U e^{loc - \widehat{loc}}] = e^{loc - \widehat{loc}} M[e^U]$$

Плотность функции распределения Гомперца $F(u; c, 0)$:

$$f(u; c, 0) = ce^{u-c(e^u-1)}, u \geq 0$$

$$\begin{aligned} M[e^U] &= \int_0^{+\infty} e^u ce^u e^{-c(e^u-1)} du = c \int_0^{+\infty} e^{2u} e^{-c(e^u-1)} du = \left[\begin{array}{l} v = e^u \\ u = \ln v \\ du = \frac{dv}{v} \end{array} \right] = \\ &= c \int_1^{+\infty} ve^{-c(v-1)} dv = \left[\begin{array}{l} w = v - 1 \\ v = w + 1 \\ dv = dw \end{array} \right] = c \int_0^{+\infty} (w+1)e^{-cw} dw = \\ &= c \left(\int_0^{+\infty} we^{-cw} dw + \int_0^{+\infty} e^{-cw} dw \right) = c \left(\frac{1}{c^2} + \frac{1}{c} \right) = \frac{1}{c} + 1 \end{aligned}$$

По вычислениям ММП $\widehat{loc} \xrightarrow{P} loc$, следовательно $e^{loc - \widehat{loc}} \xrightarrow{P} 1$.

$$M[e^{X - \widehat{loc}}] \xrightarrow{P} 1 * M[e^U] = M[e^{X - loc}] = \frac{1}{c} + 1$$

$$\frac{1}{\widehat{c}} \xrightarrow{P} M[e^{X - \widehat{loc}}] - 1 = \frac{1}{c} + 1 = \frac{1}{c}$$

$$\widehat{c} \xrightarrow{P} c$$

Следовательно, оценки параметров c и loc , вычисленные ММП состоятельные.

6.3.3 Эффективность

Оценка $\widehat{\theta}$ называется эффективной, если она имеет минимальную дисперсию. Эффективность $e(\theta)$ несмещённой оценки $\widehat{\theta}$:

$$e(\theta) = \frac{1}{nI(\theta)D\widehat{\theta}}, \text{ где } I(\theta) \text{ – информация Фишера.}$$

Информация Фишера $I(\theta)$ – мера, которая показывает сколько информации о параметре θ содержится в случайной выборке:

$$I(\theta) = M\left[\left(\frac{\partial \ln p(x; \theta)}{\partial \theta}\right)^2\right]$$

При выполнении условий регулярности эквивалентно: $-M\left[\left(\frac{\partial^2 \ln p(x; \theta)}{\partial \theta^2}\right)\right]$

Для выборки $X_1, \dots, X_n$ независимых и одинаково распределённых случайных чисел $I_n(\theta) = nI(\theta)$.

Функция плотности вероятности распределения имеет следующий вид:

$$f(x; c, loc) = \begin{cases} ce^{x-loc-c(e^{x-loc}-1)}, & x \geq loc \\ 0, & x < loc \end{cases}$$

$$\ln f(x; c, loc) = \ln c + x - loc - c(e^{x-loc} - 1)$$

$$\frac{\partial \ln f}{\partial c} = \frac{1}{c} - e^{x-loc} + 1$$

$$\frac{\partial \ln f}{\partial loc} = -1 + ce^{x-loc}$$

$$\frac{\partial \ln f}{\partial^2 c} = -\frac{1}{c^2}$$

$$\frac{\partial \ln f}{\partial^2 loc} = -ce^{x-loc}$$

Вычисляя математические ожидания (см. предыдущий пункт, в котором уже вычислялось $M[e^{X-loc}]$) получим:

$$I(c) = \frac{1}{c^2}, I(loc) = c(1 + \frac{1}{c})$$

Оценка эффективна, если она достигает нижней границы дисперсии, заданной неравенством Рао-Крамера: $D(\hat{\theta}_n) \geq \frac{1}{n I(\theta)}$

Однако, это неравенство применимо только для несмещённых оценок. Для смещённых оценок неравенство Рао-Крамера принимает вид:

$$D(\hat{\theta}_n) \geq \frac{1}{n I_n(\theta)} (1 + \delta(\theta))^2, \text{ где } \delta(\theta) = M[\hat{\theta}] - \theta - \text{смещение оценки } \hat{\theta}.$$

$$\delta(\theta) = \int \dots \int (\hat{\theta}(x_1, \dots, x_n) - \theta) p(x_1, \theta) \dots p(x_n, \theta) dx_1 \dots dx_n$$

Однако выражение $\frac{1}{n I_n(\theta)} (1 + \delta(\theta))^2$ не применимо для оценки нижней границы Рао-Крамера для параметра loc, так как область определения c зависит от параметра loc. Вместо этого в качестве нижней границы будем использовать асимптотическую дисперсию, полученную на основе теории экстремальных статистик (так как при вычислении loc ММП, ему присваивалось значение минимального элемента выборки): $\frac{1}{(nc)^2}$

Из-за сложности и нелинейности интегралов вычисление общей формулы дисперсии оценок и их смещения не представляется возможным. Поэтому на Python была написана программа, генерирующая 1000 выборок по распределению Гомперца из 300 элементов с известными заданными параметрами $\log$ и c . Затем методом моментов и методом математического правдоподобия (абсолютно таким же алгоритмом, каким вычислены оценки в пунктах 6.1 и 6.2), вычисляются оценки, их смещение, дисперсия и нижняя граница неравенства Рао-Крамера для смещенной оценки, чтобы определить являются ли они эффективными. Ознакомиться с листингом соответствующей программы можно в приложении 10.

При запуске программы получены следующие результаты:

Метод моментов:

Смещение для c : 0.109311

Смещение для $\log$: 0.000457

Дисперсия для c : 5.329235e-01

Дисперсия для $\log$: 2.392393e-05

Нижняя граница Рао-Крамера:

для c : 3.665681e-01

для $\log$: 1.243334e-07

Метод ММП:

Смещение для c : 0.075999

Смещение для $\log$: 0.000348

Дисперсия для c : 3.538943e-01

Дисперсия для $\log$: 1.291087e-07

Нижняя граница Рао-Крамера:

для c : 3.448833e-01

для $\log$: 1.243334e-07

По результатам работы программы можно сделать вывод, что оценки, полученные обоими методами можно считать эффективными. Но как видно из дисперсии оценок параметров, ММП позволяет получить более точные и надёжные результаты.

6.3.4 R-эффективность

Оценку, для которой в неравенстве Рао–Крамера достигается равенство, называют R-эффективной оценкой. Из предыдущего пункта видно, что вычисленные методом моментов и методом максимального правдоподобия оценки не позволяют достичь равенства в неравенстве Рао–Крамера. Следовательно, не являются R-эффективными.

6.4 Графическое сравнение оценок параметров

Для сравнения оценок параметров, вычисленных методом максимального правдоподобия и методом моментов, с эмпирической функцией распределения были построены графики функции распределения с вычисленными значениями параметров (см. рисунок Рисунок 12). С листингом кода, осуществляющего построение графиков можно ознакомиться в приложении 9.

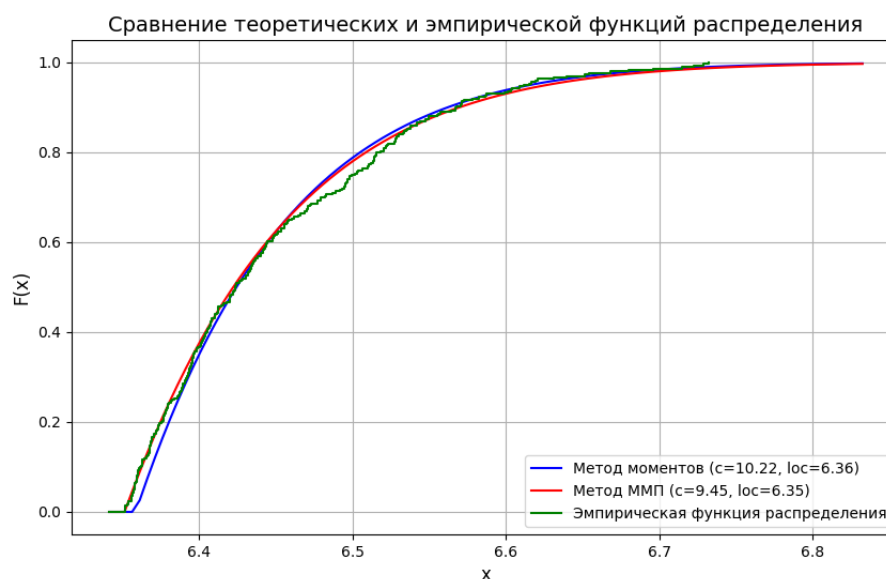


Рисунок 12 - Сравнение графиков функций распределения

Оценки параметров, полученные методом моментов и методом максимального правдоподобия, демонстрируют близкие значения, что указывает на согласованность обоих методов. Метод максимального правдоподобия оказался более точен. На графике видно, что теоретические функции распределения, построенные с использованием оценок метода

моментов и метода максимального правдоподобия, практически совпадают с эмпирической функцией распределения, что подтверждает корректность полученных параметров.

7 ПОНЯТИЕ ИНТЕРВАЛЬНЫХ ОЦЕНОК

Интервальной оценкой называют числовой интервал $(\widetilde{\theta}_n^{(1)}, \widetilde{\theta}_n^{(2)})$, определяемый по результатам выборки, относительно которого можно утверждать с определенной, близкой к единице вероятностью, что он заключает в себе значение оцениваемого параметра генеральной совокупности, т.е. $P\{\widetilde{\theta}_n^{(1)} \leq \theta \leq \widetilde{\theta}_n^{(2)}\} = \gamma$, где $\widetilde{\theta}_n^{(1)}$ и $\widetilde{\theta}_n^{(2)}$ называют также нижней и верхней границами доверительного интервала параметра θ . Вероятность γ принято называть доверительной вероятностью.

Оценим параметры распределения выборки с помощью интервальной оценки с уровнями доверия $\gamma_1 = 0,95, \gamma_2 = 0,99$. Согласно центральной предельной теореме:

$$\left\{ \theta \in \left[\bar{x} - \frac{c_\gamma \sigma}{\sqrt{n}}; \bar{x} + \frac{c_\gamma \sigma}{\sqrt{n}} \right] \right\} = \gamma;$$

Однако, эта формула применима только если $\bar{x}$ является асимптотически нормальной оценкой θ , а σ известно или может быть оценено. В нашем случае при вычислениях оценки параметра лос методом моментов $\widehat{los}$ численно зависит от $M[U]$, которое, в свою очередь, численно зависит от параметра c и вычислялось программно. Соответственно σ для параметра лос не задано в явном виде.

Оценки, полученные ММП являются асимптотически нормальными (гарантируется центральной предельной теоремой). Их дисперсию можно оценить с помощью информации Фишера:

$$I(\theta) = M\left[\left(\frac{\partial \ln p(x; \theta)}{\partial \theta}\right)^2\right]$$

При выполнении условий регулярности эквивалентно: $-M\left[\frac{\partial^2 \ln p(x; \theta)}{\partial \theta^2}\right]$

Для выборки из n наблюдений, общая информация Фишера равна $nI(\theta)$. Тогда асимптотическая дисперсия оценки выражается как:

$$D[\hat{\theta}] \approx \frac{1}{nI(\theta)}$$

7.1 Интервальная оценка параметра c

Для параметра c информация Фишера уже вычислялась в предыдущем пункте: $I(c) = \frac{1}{c^2}$. Тогда асимптотическая дисперсия $D[\hat{c}] \approx \frac{c^2}{n} = \sigma^2$. Теперь можем применить:

$$\{c \in [\hat{c} - C_\gamma \sigma; \hat{c} + C_\gamma \sigma]\} = \left\{c \in \left[\hat{c} - C_\gamma \frac{\hat{c}}{\sqrt{n}}; \hat{c} + C_\gamma \frac{\hat{c}}{\sqrt{n}}\right]\right\} = \gamma = 2\Phi_0(C_\gamma),$$

где C_γ – квантиль нормального распределения, $\hat{c}$ – оценка параметра c , вычисленная методом максимального правдоподобия.

Для $\gamma = 0.95$ $\Phi_0(C_{0.95}) = 0,475$ и по таблице значений функции Лапласа $C_{0.95} = 1.96$.

Для $\gamma = 0.99$ $\Phi_0(C_{0.95}) = 0,495$ и по таблице значений функции Лапласа $C_{0.95} = 2.58$.

Подставляя численные значения получим интервалы:

$$c \in [8.38359; 10.52307] \text{ с уровнем доверия } \gamma_1 = 0.95$$

$$c \in [8.0452; 10.86146] \text{ с уровнем доверия } \gamma_2 = 0.99$$

7.2 Интервальная оценка параметра loc

Оценка параметра loc методом максимального правдоподобия выглядела следующим образом: $\widehat{loc} = \min(x_i) = x_{(1)}$. loc является параметром сдвига и ограничивает область определения плотности распределения, поэтому будем строить для него односторонний доверительный интервал.

Пусть $t \geq loc$, тогда:

$$\begin{aligned} P\{x_{(1)} > t\} &= P\{x_1 > t, \dots, x_n > t\} = (P\{x_1 > t\})^n = \left(\int_t^{+\infty} f(x; c, loc) dx\right)^n \\ &= [u = x - loc] = \left(\int_{t-loc}^{+\infty} f(u; c, 0) du\right)^n \end{aligned}$$

Подставим функцию плотности распределения Гомперца:

$$P\{x_1 > t\} = \int_t^{+\infty} ce^{x-loc-c(e^{x-loc}-1)} dx = [u = x - loc] = \int_{t-loc}^{+\infty} ce^{u-c(e^u-1)} du$$

Для вычисления одностороннего доверительного интервала хотим получить следующее выражение:

$$P\{x_{(1)} - z \leq loc \leq x_{(1)}\} = \gamma$$

$$P\{x_{(1)} > t\} = \left(\int_{t-loc}^{+\infty} ce^{u-c(e^u-1)} du \right)^n$$

Примем $z = t - loc$:

$$P\{x_{(1)} > z + loc\} = \left(\int_z^{+\infty} f(u; c, 0) du \right)^n$$

Примем:

$$\left(\int_z^{+\infty} f(u; c, 0) du \right)^n = 1 - \gamma$$

$$\int_z^{+\infty} f(u; c, 0) du = (1 - \gamma)^{\frac{1}{n}}$$

$$\int_z^{+\infty} ce^{u-c(e^u-1)} du = (1 - \gamma)^{\frac{1}{n}}$$

Решив это уравнение, найдем z , а затем получим доверительный интервал:

$$loc \in [x_{(1)} - z, x_{(1)}] \text{ с уровнем доверия } \gamma.$$

Интеграл не имеет простого аналитического решения, поэтому он численно вычислялся программно с помощью библиотеки Python `scipy.integrate`. Решение уравнения также производилось программно с помощью библиотеки Python `scipy.optimize`. С листингом кода, реализующим вычисления можно ознакомиться в приложении 11. В результате выполнения программы получены следующие доверительные интервалы:

$$loc \in [6.350253, 6.351309] \text{ с уровнем доверия } \gamma_1 = 0.95$$

$$loc \in [6.349686, 6.351309] \text{ с уровнем доверия } \gamma_2 = 0.99$$

Точечная оценка параметра c находится внутри обоих интервалов, что подтверждает её состоятельность и эффективность. Довольно большая ширина интервалов для параметра c отражает большую чувствительность оценки формы распределения Гомперца к данным, что связано с нелинейной природой параметра c и его влиянием на хвосты распределения. Точечная оценка параметра loc (вычисленная ММП) находится на верхней границе обоих интервалов, что ожидаемо для одностороннего интервала, построенного на основе $\min(x_i)$. Это подтверждает корректность используемого метода. Ширина интервала для параметра loc очень мала, что отражает высокую точность оценки loc как минимального элемента выборки при большом n .

8 ПОНЯТИЕ СТАТИСТИЧЕСКИХ КРИТЕРИЕВ

Статистическая гипотеза – априорное предположение о виде теоретической функции $F(x)$.

Задача проверки статистических гипотез – по выборке $(x_1, \dots, x_n)$ принять либо основную гипотезу H_0 , либо альтернативную гипотезу H_1 .

Статистический критерий – правило, позволяющее только по выборке $(x_1, \dots, x_n)$ принять либо H_0 , либо H_1 .

Статистика критерия – случайная величина $K = K(x_1, \dots, x_n)$, которая служит для проверки гипотезы.

Значение $K_{\text{набл}} = K_{\text{набл}}(x_1, \dots, x_n)$ этой случайной величины – наблюдаемое значение критерия.

8.1 Гипотезы о параметрах распределения

По лемме Неймана-Пирсона среди всех критериев с заданным уровнем значимости α критерий отношения правдоподобия является наиболее мощным, поэтому будем использовать его.

Т.е. по имеющейся выборке $(x_1, \dots, x_n) \sim F(c, loc)$ – распределение Гомперца для фиксированного уровня значимости $\alpha = 0,05$ будем проверять сложные гипотезы:

1) $\frac{H_0:\{c=\widehat{c}_1\}}{H_1:\{c\neq\widehat{c}_1\}}$, где $\widehat{c}_1$ – оценка параметра c , вычисленная методом моментов.

2) $\frac{H_0':\{c=\widehat{loc}_1\}}{H_1':\{c\neq\widehat{loc}_1\}}$, где $\widehat{loc}_1$ – оценка параметра loc , вычисленная методом моментов.

3) $\frac{H_0^*:\{c=\widehat{c}_2\}}{H_1^*:\{c\neq\widehat{c}_2\}}$, где $\widehat{c}_2$ – оценка параметра c , вычисленная методом максимального правдоподобия.

4) $\frac{H_0^{**}:\{c=\widehat{loc}_2\}}{H_1^{**}:\{c\neq\widehat{loc}_2\}}$, где $\widehat{loc}_2$ – оценка параметра loc , вычисленная методом максимального правдоподобия.

Распределение Гомперца имеет следующую функцию плотности вероятности:

$$f(x; c, loc) = \begin{cases} ce^{x-loc-c(e^{x-loc}-1)}, & x \geq loc \\ 0, & x < loc \end{cases}$$

Функция правдоподобия:

$$L(x_1, \dots, x_n; c, loc) = \prod_{i=1}^n ce^{x_i-loc-c(e^{x_i-loc}-1)} = c^n e^{\sum_{i=1}^n (x_i-loc)-c \sum_{i=1}^n (e^{x_i-loc}-1)}$$

Подставив численные значения оценок, вычисленных методом максимального правдоподобия (так как в ММП как раз получали значения, при которых функция правдоподобия достигает максимума) получим:

$$L^*(x_1, \dots, x_n, \widehat{c}_1, \widehat{loc}_1) = 5.449861 * 10^{164}$$

$$\ln L^*(x_1, \dots, x_n, \widehat{c}_1, \widehat{loc}_1) = 379.319545$$

$$l = \ln L(x_1, \dots, x_n; c, loc) = n \ln c + \sum_{i=1}^n (x_i - loc) - c \sum_{i=1}^n (e^{x_i-loc} - 1)$$

При вычислении функции правдоподобия, для корректности взятия логарифма отбрасывались значения выборки, превышающие значение loc .

а) $\frac{H_0:\{c=\widehat{c}_1\}}{H_1:\{c\neq\widehat{c}_1\}}$

Найдем $\widehat{\theta}_0 = (\widehat{c}_1^{(0)}, \widehat{loc}_1^{(0)})$ – максимизирует функцию L на множестве θ_0 ($c = \widehat{c}_1$).

$$\frac{\partial \ln L(x_1, \dots, x_n, \widehat{c}_1, loc)}{\partial loc} = -n + \widehat{c}_1 \sum_{i=1}^n e^{x_i - loc} = 0$$

Отсюда сложно выразить формулу для loc , решение уравнения - значение $(loc)_0 = 6.832307$ вычисляется программно, подставляя значение $\widehat{c}_1$, вычисленное методом моментов. Получаем $\widehat{\theta}_0 = (\widehat{c}_1, (loc)_0)$.

Вычисляем численное значение:

$$\ln L_0^*(x_1, \dots, x_n, \widehat{c}_1, (loc)_0) = 1438.017912$$

Вычисляем отношение правдоподобия:

$$\Lambda = \Lambda(x_1, x_2, \dots, x_n) = \frac{L^*(x_1, x_2, \dots, x_n; c, loc)}{L_0^*(x_1, x_2, \dots, x_n; c, loc)}$$

Критическая область критерия отношения правдоподобия – все точки $\lambda = \lambda(x_1, \dots, x_n) : \Lambda > C$, где C – критическое значение. Пусть $\lambda = 2 \ln \Lambda$, если $\lambda = \lambda(x_1, \dots, x_n) < C$, то принимаем H_0 , иначе – принимаем H_1 .

При условии, что $\vec{\theta} \in \theta_0$, при $n \rightarrow \infty$ (асимптотическое свойство метода отношения правдоподобия):

$$\lambda(x_1, \dots, x_n) \sim \chi^2(k - m)$$

где k – размерность пространства общего множества, m – размерность множества принятия гипотезы H_0 , т. е. $m = 1$, $k = 2$. Таким образом, получаем критическое значение $C = \chi_{0,05}^2 = 3.841459$.

Получаем $\lambda = 2 \ln \Lambda = 2117.396733 \gg 3.841459$. Следовательно, не принимаем гипотезу $H_0: \{c = \widehat{c}_1\}$, где $\widehat{c}_1 = 10.221538$, вычисленная методом моментов.

$$b) \begin{matrix} H'_0: \{c = \widehat{loc}_1\} \\ H'_1: \{c \neq \widehat{loc}_1\} \end{matrix}$$

Найдем $\widehat{\theta}_0 = (\widehat{c}_1^{(0)}, \widehat{loc}_1^{(0)})$ – максимизирует функцию L на множестве θ_0 ($loc = \widehat{loc}_1$).

$$\frac{\partial \ln L(x_1, \dots, x_n, \widehat{c}_1, loc)}{\partial c} = \frac{n}{c} - \sum_{i=1}^n (e^{x_i - loc} - 1) = 0$$

$$c = \frac{n}{\sum_{i=1}^n (e^{x_i - loc} - 1)}$$

Программно вычисляем значение $(c)_0$, подставляя значение $\widehat{loc}_1$, вычисленное методом моментов. Получили $(c)_0 = 9.525233$

Получаем $\widehat{\theta}_0 = ((c)_0, \widehat{loc}_1)$. Вычисляем численное значение:

$$L_0^*(x_1, \dots, x_n, (c)_0, \widehat{loc}_1) = 1.588962 * 10^{164}$$

$$\ln L_0^*(x_1, \dots, x_n, (c)_0, \widehat{loc}_1) = 378.087037$$

Вычисляем отношение правдоподобия:

$$\Lambda = \Lambda(x_1, x_2, \dots, x_n) = \frac{L^*(x_1, x_2, \dots, x_n; c, loc)}{L_0^*(x_1, x_2, \dots, x_n; c, loc)} = 3.429823$$

Получаем $\lambda = 2 \ln \Lambda = 2.465018 < 3.841459 = C$. Следовательно, принимаем гипотезу $H'_0: \{loc = \widehat{loc}_1\}$, где $\widehat{loc}_1 = 6.358577$, вычисленная методом моментов. Программа, осуществляющая описанные выше действия представлена в приложении 12.

$$c) \begin{matrix} H_0^*: \{c = \widehat{c}_2\} & H_0^{**}: \{loc = \widehat{loc}_2\} \\ H_1^*: \{c \neq \widehat{c}_2\} & H_1^{**}: \{loc \neq \widehat{loc}_2\} \end{matrix}$$

Поскольку (по определению метода максимального правдоподобия) $\widehat{c}_2, \widehat{loc}_2$ – значения, которые максимизируют функцию правдоподобия L , то при проверке этих гипотез критерием отношения правдоподобия мы всегда будем получать $\Lambda(x_1, x_2, \dots, x_n) = \frac{L^*(x_1, x_2, \dots, x_n; c, loc)}{L_0^*(x_1, x_2, \dots, x_n; c, loc)} = 1$; $2 \ln \Lambda = 0$ и всегда принимать гипотезу. Т.е. для оценок, вычисленных ММП критерий не эффективен. Другим словами, гипотезы H_0^* и H_0^{**} будут приниматься всегда. Поэтому при проверке гипотез об оценках ММП методом отношений правдоподобий, ошибка второго рода β (вероятность принять H_0 когда верна H_1) будет равна 1, а мощность критерия, соответственно, $1 - \beta = 0$.

При проверке гипотез из-за наличия неполной информации могут быть допущены ошибки двух видов (Рисунок 13). Ошибка первого рода заключается в том, что верная нулевая гипотеза H_0 отвергается и принимается конкурирующая ложная гипотеза H_1 . Ошибка второго рода заключается в том, что ложная гипотеза H_0 принимается, хотя на самом деле верна конкурирующая гипотеза H_1 .

		Принята по результатам расчета	Вывод
Верна на самом деле	H_0	H_0	Вероятность правильного решения $1 - \alpha$
		H_1	Ошибка первого рода (ложная тревога, ложное срабатывание, ложноположительное срабатывание). Вероятность ошибки – α
	H_1	H_1	Вероятность правильного решения $\beta' = 1 - \beta$
		H_0	Ошибка второго рода (пропуск цели, пропуск события, ложноотрицательное срабатывание). Вероятность ошибки – β .

Рисунок 13 – Ошибки 1-го и 2-го рода

Вероятность ошибки первого рода равна уровню значимости, который был принят $\alpha = 0,05$, ошибка второго рода равна $\beta = 1 - P\{\lambda(x_1, x_2, \dots, x_n) \geq C\} = P\{2 \ln \Lambda < 3.841459 | H_1\}$. В данном случае, альтернативные гипотезы были сложными, поэтому нельзя численно выразить ошибку второго рода. Для приблизительной оценки ошибки второго рода примем простую альтернативную гипотезу: $\frac{H_0: \{c=\hat{c}_1\}}{H_1: \{c=\hat{c}_2\}}$, где $\hat{c}_1$ – оценка параметра c , вычисленная методом моментов, $\hat{c}_2$ – методом максимального правдоподобия. ММП оценки асимптотически нормальны, поэтому $2 \ln \Lambda$ будет иметь нецентральное χ^2 -распределение: $2 \ln \Lambda \sim \chi^2_1(\delta)$, где δ – параметр нецентральности, зависящий от разницы между $\hat{c}_1$ и $\hat{c}_2$. Для

одного параметра (фиксируем c , максимизируем по $\log$), параметр нецентральности:

$$\delta \approx nI(\hat{c}_2)(\hat{c}_1 - \hat{c}_2)^2,$$

где $I(c) = \frac{1}{c^2}$ – информация Фишера для c (вычислялась в предыдущих пунктах). Подставив численные значения, получим $\delta \approx 1.98111$.

Соответственно, ошибка второго рода $\beta = P\{\chi_1^2(\delta) < 3.841459\}$. Вычислялась программно с помощью встроенной функции `ncx2` библиотеку `scipy.stats`, вычисляющей значения нецентрального χ^2 -распределения. На листинге Листинг 1 представлен соответствующий код. Получено следующее численное значение: $\beta = 0.7093$.

Листинг 1 - Вычисление ошибки второго рода

```
from scipy.stats import ncx2
delta = 1.98111
beta = ncx2.cdf(3.841459, df=1, nc=delta)
print(f"Ошибка второго рода для c: {beta:.4f}")
```

Проведем аналогичные вычисления для вычисления значения ошибки второго рода оценки параметра $\log$. Как и в пункте об исследовании оценок на эффективность, в качестве $I(\log)$ примем $(nc)^2$.

$$\delta \approx I(\widehat{\log c}_2)(\widehat{\log c}_1 - \widehat{\log c}_2)^2 = 424.8562$$

$$\beta = P\{\chi_1^2(\delta) < 3.841459\} \approx 0$$

Можно сделать вывод, что критерий отношения правдоподобия является достаточно мощным для проверки гипотез об оценках, вычисленных не методом максимального правдоподобия. В случае оценок ММП он становится тривиальным и не эффективным.

8.2 Гипотезы о виде распределения

8.2.1 Критерий Колмогорова

Будем проверять следующую нулевую гипотезу:

H_0 (простая): $F(x) = \hat{F}(x)$, т.е. выборка принадлежит распределению Гомперца.

Альтернативная гипотеза: H_1 (сложная): $F(x) \neq \hat{F}(x)$.

Критерий Колмогорова заключается в оценивании максимального расстояния между значениями функции распределения выборки и теоретической функцией распределения:

$$D = \rho = \sup |F(x) - \hat{F}_n(x)|$$

Статистика D сравнивается с критическим значением $D_{\text{крит}}$, которое при уровне значимости $\alpha=0,05$ и заданном количестве элементов выборки ($n=300$) принимает значение $D_{\text{крит}} = \frac{1.36}{\sqrt{300}} = 0.0785$ (вычисляется по таблице Колмогорова-Смирнова). Если $D < D_{\text{крит}}$, то нулевая гипотеза принимается.

Была реализована программа, вычисляющая статистику критерия Колмогорова, с листингом которой можно ознакомиться в приложении 13. На рисунке Рисунок 14 представлены промежуточные программные вычисления (вывод только для каждого 10-го элемента выборки для компактности).

Промежуточные вычисления (каждый 10-й элемент):			
x_i	$F_n(x_i)$	$F(x_i)$	$ F_n - F $
6.351309	0.003333	0.000000	0.003333
6.356037	0.036667	0.043812	0.007146
6.359038	0.070000	0.070722	0.000722
6.362242	0.103333	0.098703	0.004630
6.367591	0.136667	0.143738	0.007071
6.370390	0.170000	0.166491	0.003509
6.375684	0.203333	0.208049	0.004716
6.379665	0.236667	0.238066	0.001399
6.387987	0.270000	0.297540	0.027540
6.392299	0.303333	0.326686	0.023353
6.395457	0.336667	0.347338	0.010671
6.400704	0.370000	0.380394	0.010394
6.405046	0.403333	0.406602	0.003269
6.409961	0.436667	0.435062	0.001605
6.419868	0.470000	0.488723	0.018723
6.423649	0.503333	0.507960	0.004627
6.432243	0.536667	0.549280	0.012614
6.438222	0.570000	0.576151	0.006151
6.444604	0.603333	0.603230	0.000103
6.455221	0.636667	0.644833	0.008166
6.468567	0.670000	0.691516	0.021516
6.481973	0.703333	0.732742	0.029409
6.492999	0.720000	0.762831	0.042831
6.496464	0.736667	0.771630	0.034963
6.509632	0.770000	0.802431	0.032431
6.518029	0.803333	0.820046	0.016713
6.528965	0.836667	0.840843	0.004176
6.543991	0.870000	0.865851	0.004149
6.567910	0.903333	0.898355	0.004978
6.601376	0.936667	0.931833	0.004833
6.639139	0.970000	0.957275	0.012725

Максимальная разница $ F_n - F = 0.042831$ достигнута при $x = 6.492999$			
$F_n(x) = 0.720000$, $F(x) = 0.762831$			

Рисунок 14 - Промежуточные вычисления для критерия Колмогорова

Полученное численное значение: $D = 0.042831 < 0.0785 = D_{\text{крит}}$, следовательно, принимаем гипотезу о том, что выборка действительно принадлежит распределению Гомперца.

8.2.2 Критерий χ^2

Будем проверять следующую нулевую гипотезу:

H_0 (простая): $F(x) = \hat{F}(x)$, т.е. выборка принадлежит распределению Гомперца.

Альтернативная гипотеза: H_1 (сложная): $F(x) \neq \hat{F}(x)$.

Критерий χ^2 в общем случае реализуется следующим образом:

- 1) Область значений случайной величины разбивается на k интервалов: $(-\infty; d_1), [d_1, d_2), \dots, [d_{k-1}, +\infty)$;
- 2) Определяются теоретические вероятности $p_i = F(d_i) - F(d_{i-1})$ попадания в интервал $[d_{i-1}, d_i)$ и числа n_i элементов выборки, попавших в эти интервалы.
- 3) Вычисляется статистика критерия: $\chi_{\text{набл}}^2 = \sum_{i=1}^k \frac{(n_i - np_i)^2}{np_i}$
- 4) Если статистика критерия меньше критического значения $\chi_{\text{кр}}^2 = \chi^2(k - 1 - r)$, то нулевая гипотеза принимается. (r – количество параметров, которые оцениваются по выборке).

Количество интервалов для разбиения было выбрано согласно формуле Стерджесса:

$$k = [\log_2 n + 1] = [\log_2 300 + 1] = 9$$

Соответственно, критическое значение вычисляется для 6 (9-1-2) степеней свободы, так как по выборке оценивались два параметра – c и $\log$.

Была реализована программа, выполняющая описанные выше действия, с листингом которой можно ознакомиться в приложении 13. На рисунке Рисунок 15 представлены получившиеся вероятности попадания в интервалы:

Интервал	Вероятность p_i	Ожидаемая частота $n \cdot p_i$	Наблюдаемая частота
[6.3513, 6.3936]	0.335540	100.661939	95
[6.3936, 6.4360]	0.230689	69.206566	73
[6.4360, 6.4783]	0.155772	46.731531	40
[6.4783, 6.5206]	0.103227	30.968226	35
[6.5206, 6.5630]	0.067080	20.124019	24
[6.5630, 6.6053]	0.042709	12.812636	16
[6.6053, 6.6476]	0.026619	7.985558	8
[6.6476, 6.6900]	0.016225	4.867614	4
[6.6900, 6.7323]	0.009663	2.899047	5

Рисунок 15 – Распределение частоты попадания в интервалы для критерия χ^2

При запуске программы получены следующие результаты:

$$\chi^2_{\text{набл}} = 5.237639 < 12.591587 = \chi^2_{\text{кр}} = \chi^2(6)$$

Следовательно, нулевая гипотеза о том, что выборка принадлежит распределению Гомперца принимается.

На основании проведённого анализа с использованием критерия Колмогорова и критерия χ^2 установлено, что гипотеза о принадлежности выборки распределению Гомперца с заданными параметрами принимается. Критерий Колмогорова показал, что максимальное отклонение эмпирической функции распределения от теоретической находится в пределах допустимого; критерий χ^2 указал на отсутствие значимых различий между наблюдаемыми и ожидаемыми частотами. Таким образом, оба критерия свидетельствуют о том, что выборка не противоречит заданному теоретическому распределению.

8.3 Гипотезы об однородности выборок

Распределение Гомперца близко к виду распределения Вейбулла, гамма-распределению, логнормальному распределению и к экспоненциальному распределению со сдвигом.

Далее задания будут выполняться для гамма-распределения. Функция плотности гамма-распределения выглядит следующим образом:

$$f(x; k, \theta) = \frac{1}{\Gamma(k)\theta^k} x^{k-1} e^{-\frac{x}{\theta}},$$

где α – параметры формы, θ – параметр масштаба.

Функция распределения не имеет аналитического выражения в общем виде и вычисляется через нижнюю неполную гамма-функцию:

$$F(x; k, \theta) = \frac{1}{\Gamma(k)} \gamma\left(k, \frac{x}{\theta}\right).$$

На рисунке Рисунок 16 представлен график функции гамма-распределения.

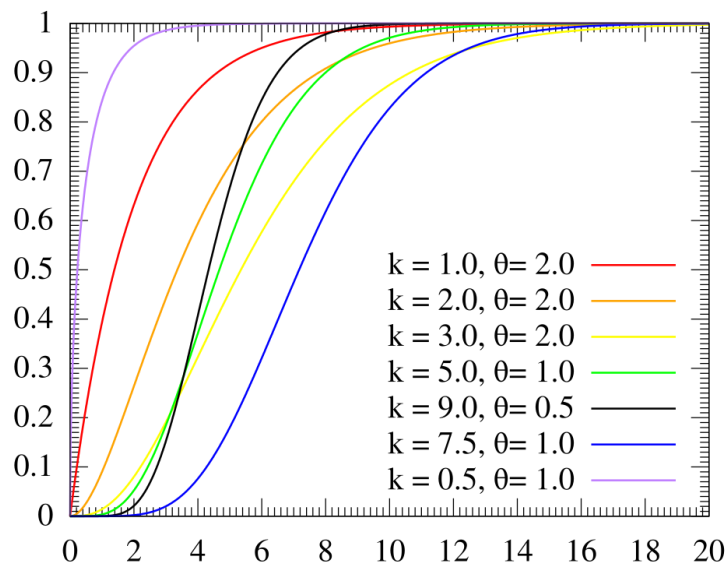


Рисунок 16 - График функции гамма-распределения

Так как далее будем проверять гипотезу об однородности выборок, то аналогично распределению Гомперца будем использовать гамма-распределение с параметром смещения loc :

$$f(x; k, \theta, loc) = \frac{1}{\Gamma(k)\theta^k} (x - loc)^{k-1} e^{-\frac{x-loc}{\theta}},$$

$$F(x; k, \theta, loc) = \frac{1}{\Gamma(k)} \gamma\left(k, \frac{x - loc}{\theta}\right).$$

Для генерации выборки методом обратного преобразования необходимо выразить обратную к гамма-распределению функцию. Однако, обратная неполная гамма-функция не имеет аналитического решения, поэтому для генерации выборки X_2 , была использована встроенная функция Python `gamma.pdf()` библиотеки `scipy.stats.gamma`.

Необходимо определить какие параметры гамма-распределения нужно использовать при генерации выборки X_2 . Значение параметра loc примем одинаковым с оценкой параметра loc выборки X_2 , вычисленной методом максимального правдоподобия.

$$\text{loc} = \min(X_1) = 6.351309$$

Известны основные статистики гамма-распределения:

- Математическое ожидание $k\theta$ (со смещением $k\theta + \text{loc}$)
- Дисперсия $k\theta^2$.

Приравняем к уже вычисленным во втором пункте статистикам выборки X_1 , решим систему и получим следующие значения:

$$k = 1.36, \quad \theta = 0.071.$$

Были сформулированы следующие гипотезы:

- $H_0\{F_1(x), F_2(x) \sim F(x; c, \text{loc})\}$ – нулевая;
- $H_1: \{F_1(x) \sim F(x; c, \text{loc}), F_2(x) \sim G(x; k, \theta, \text{loc})\}$ – альтернативная;

где $F_1(x)$ – эмпирическая функция распределения, построенная по изначальной выборке X_1 , $F_2(x)$ – эмпирическая функция распределения, построенная по сгенерированной выборке X_2 , $F(x; c, \text{loc})$ – теоретическая функция распределения Гомперца, $G(x; k, \theta, \text{loc})$ – теоретическая функция гамма-распределения.

Для проверки сформулированной нулевой гипотезы однородности была выбран критерий Колмогорова-Смирнова, в котором для двух выборок $(x_1, \dots, x_k)$ и $(y_1, \dots, y_n)$ статистика представляет собой следующее выражение:

$$\rho = \sqrt{\frac{kn}{k+n}} \sup |\widehat{F}_1(z) - \widehat{F}_2(z)|,$$

где $\widehat{F}_1(z), \widehat{F}_2(z)$ – эмпирические функции распределения выборок $(x_1, \dots, x_k)$ и $(y_1, \dots, y_n)$ соответственно.

При $k, n \leq 20$ критические значения d принимаются по таблицам (нулевая гипотеза H_0 принимается, если $\rho \leq d$). При больших объемах

выборки (как в нашем случае, обе выборки содержат 300 элементов) статистика ρ имеет распределение Колмогорова, уровень значимости определяется как $\alpha = 1 - K(d)$:

$$\forall d > 0 \lim_{k,n \rightarrow \infty} P\{\rho \leq d\} = K(d).$$

При уровне значимости $\alpha = 0.05$:

$$K(d) = 1 - \alpha = 0.95 \Rightarrow d = 1.36$$

Эмпирические функции $\widehat{F}_1(x), \widehat{F}_2(x)$, построенные по выборкам X_1 и X_2 являются кусочно-заданными (т.е. увеличиваются на $\frac{1}{n_1}$ или $\frac{1}{n_2}$ только в точках выборок X_1 или X_2 , соответственно). Чтобы найти максимальную разницу $|\widehat{F}_1(x) - \widehat{F}_2(x)|$, нужно оценить ее во всех точках, где хотя бы одна из функций изменяет свое значение, т.е. в точках из обеих выборок. Для этого выборки X_1 и X_2 были объединены в одну отсортированную последовательность, которая позволяет перебрать все такие точки. Для каждой точки x из объединенной выборки вычисляется $F_{n_1}(x)$ - доля элементов X_1 меньших x и $F_{n_2}(x)$ доля элементов X_2 меньших x . С помощью встроенного метода Python `np.interp` значения F_{n_1} и F_{n_2} интерполируются во всех точках разрыва эмпирических функций, чтоб конкретно вычислять разницу между функциями. С полным листингом программы, реализующей описанные действия можно ознакомиться в приложении 14.

Программно вычисленное значение статистики:

$$\rho = 1.003712 < 1.36 = d.$$

Следовательно, принимаем нулевую гипотезу H_0 об однородности выборок X_1 и X_2 .

На рисунке Рисунок 17 представлено сравнение эмпирических функций, построенных по выборкам X_1 и X_2 .

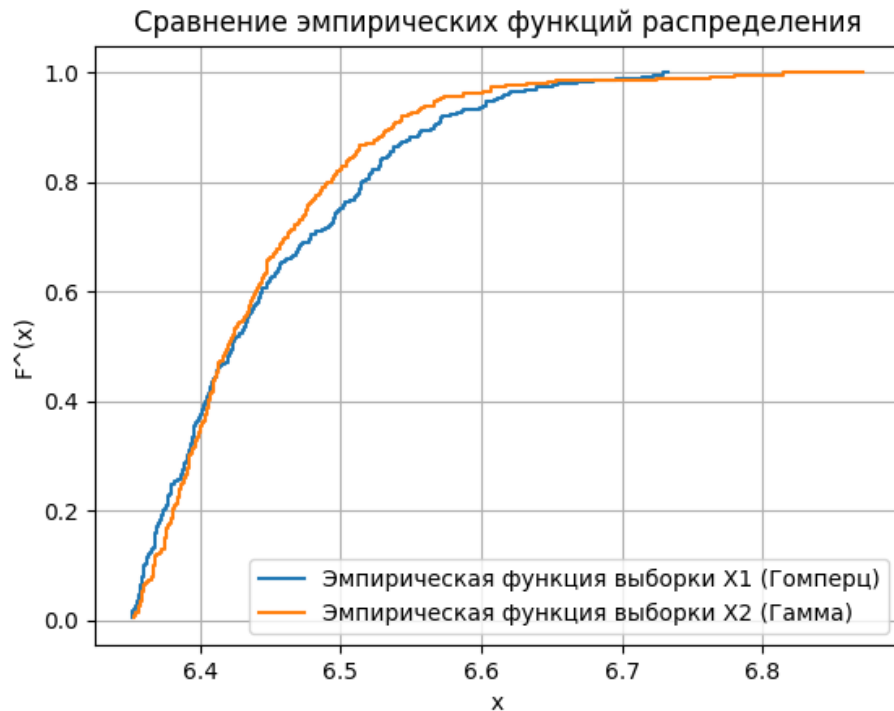


Рисунок 17 - Сравнение эмпирических функций распределения выборок X_1 и X_2 .

Мощность критерия – вероятность правильно отклонить нулевую гипотезу, когда верна альтернативная.

$$\beta' = 1 - \beta = P\{(x_1, \dots, x_2) \in Z_1 | H_1\},$$

где Z_1 – множество отклонения гипотезы H_0 . Для вычисления мощности критерия была написана программа, с полным кодом которой можно ознакомиться в приложении 15. В коде симулируется 1000 пар выборок по 300 элементов из распределения Гомперца и гамма-распределения с указанными выше параметрами. Критерий Колмогорова-Смирнова применяется к каждой из пары выборок и подсчитывается доля корректных отвержений нулевой гипотезы. По результатам выполнения программы получено следующее значение мощности критерия:

$$\beta' = 1 - \beta = 0.15$$

Можно сделать вывод, критерий Колмогорова-Смирнова в данном случае обладает довольно низкой мощностью, что указывает на его ограниченную способность различать распределение Гомперца и гамма-

распределение при подобранных параметрах и размере выборок, что может быть связано с небольшой разницей между их теоретическими функциями.

9 ЗАКЛЮЧЕНИЕ

В ходе курсовой работы было подробно исследовано распределение Гомперца с точки зрения математической статистики. Были вычислены основные статистики выборки из варианта, построена эмпирическая функция распределения и гистограммы по подвыборкам разного размера.

Для оценки параметров распределения Гомперца были применены метод моментов и метод максимального правдоподобия, что позволило получить следующие значения точечных оценок:

- Методом моментов:

$$\hat{c} = 10.221538, \widehat{loc} = 6.358577$$

- Методом максимального правдоподобия:

$$\hat{c} = 9.453330, \widehat{loc} = 6.351309$$

Полученные точечные оценки покрывают предположения о параметрах распределения, выдвинутые на основе анализа построенных по выборке графиков эмпирической функции распределения и гистограммы.

Интервальные оценки, вычисленные на основе центральной предельной теоремы и асимптотической нормальности оценок, вычисленных методом максимального правдоподобия, подтвердили надежность полученных результатов. Были получены следующие доверительные интервалы:

$$c \in [8.38359; 10.52307] \text{ с уровнем доверия } \gamma_1 = 0.95$$

$$c \in [8.0452; 10.86146] \text{ с уровнем доверия } \gamma_2 = 0.99$$

$$loc \in [6.350253, 6.351309] \text{ с уровнем доверия } \gamma_1 = 0.95$$

$$loc \in [6.349686, 6.351309] \text{ с уровнем доверия } \gamma_2 = 0.99$$

Эти интервалы покрывают соответствующие точечные оценки, вычисленные методом максимального правдоподобия, что подтверждает их состоятельность. Оценки, вычисленные методом моментов, также укладываются в интервальные оценки с учетом их большего разброса, что

указывает на их корректность и меньшую точность по сравнению с ММП оценками.

Гипотезы о параметрах распределения проверялись с помощью критерия отношения правдоподобия, что позволило отклонить гипотезу $H_0: \{c = \hat{c}_1\}$ и принять гипотезу $H_0: \{loc = \widehat{loc}_1\}$ для оценок, вычисленных методом моментов. Гипотезы для оценок ММП были приняты, так как критерий отношения правдоподобия дает для ММП оценок тривиальный результат, что говорит о его низкой эффективности относительно них.

Гипотеза о принадлежности выборки распределению Гомперца была принята по результатам проверки критериями Колмогорова и χ^2 , что подтверждает корректность и адекватность используемой выборки.

Гипотеза об однородности исходной выборки и выборки, сгенерированной из гамма-распределения методом обратного преобразования, была принята, что говорит о схожести распределений. Однако мощность критерия составила всего 0.015, что указывает также на его ограниченную чувствительность к различиям между этими распределениями.

В ходе курсовой работы мы столкнулись со множеством трудностей, касающихся сложной аналитической формы функции распределения Гомперца. Их удалось решить благодаря программным методам численного вычисления сложных интегралов и уравнений с использованием языка программирования Python.

СПИСОК ИСТОЧНИКОВ

1. Гомперц Б. О природе функции, выражающей закон человеческой смертности, и о новом способе определения стоимости пожизненных рент // *Philosophical Transactions of the Royal Society of London*. – 1825. – Vol. 115. – С. 513–583.
2. Гаврилов Л.А., Гаврилова Н.С. Биология продолжительности жизни: количественные аспекты. – М.: Наука, 1991. – 46 с.
3. Миссов Т.И., Ленарт. А. Применение распределения Гомперца для анализа данных о смертности на поздних возрастах // *Demography*. – 2011. – Vol. 48, № 3. – С. 1031–1050.

ПРИЛОЖЕНИЕ 1

Листинг кода, вычисляющего основные статистики выборки X:

```
import csv
# Чтение выборки из файла .csv
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X
# Сумма элементов выборки
def sum_x(X):
    return sum(x for x in X)
# Выборочное среднее
def mean_x(X):
    return sum_x(X) / len(X)
# Медиана
def median(X):
    sorted_X = sorted(X)
    n = len(X)
    if n % 2 == 0:
        return (sorted_X[n//2 - 1] + sorted_X[n//2]) / 2
    else:
        return sorted_X[n//2]
# Мода
def mode(X):
    freq = {}
    for x in X:
        freq[x] = freq.get(x, 0) + 1
    max_freq = max(freq.values())
    modes = [x for x, f in freq.items() if f == max_freq]
    return modes if max_freq > 1 else "Моды нет"
# Размах выборки
def range_x(X):
    return max(X) - min(X)
# Смещенная дисперсия
def variance_biased(X):
    mean = mean_x(X)
    return sum((x - mean) ** 2 for x in X) / len(X)
# Несмещенная дисперсия
def variance_unbiased(X):
    mean = mean_x(X)
    return sum((x - mean) ** 2 for x in X) / (len(X) - 1)
# Выборочный начальный момент k-го порядка
def moment_initial(X, k):
    return sum(x ** k for x in X) / len(X)
# Выборочный центральный момент k-го порядка
def moment_central(X, k):
    mean = mean_x(X)
    return sum((x - mean) ** k for x in X) / len(X)
```

```
# Чтение из файла и вывод результата
X = read_csv('sample.csv')
print("1. Сумма элементов выборки:", sum_x(X))
print("2. Выборочное среднее:", mean_x(X))
print("3. Медиана:", median(X))
print("4. Мода:", mode(X))
print("5. Размах выборки:", range_x(X))
print("6. Смещённая дисперсия:", variance_biased(X))
print("7. Несмещённая дисперсия:", variance_unbiased(X))
print("8. Начальный момент 3-го порядка:", moment_initial(X, 3))
print("9. Центральный момент 3-го порядка:", moment_central(X,
3))
```


ПРИЛОЖЕНИЕ 2

Листинг кода, для построения эмпирической функции распределения:

```
import csv
import random
import matplotlib.pyplot as plt
# Чтение выборки из файла .csv
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X
# Функция для ЭФР
def cdf(X):
    sorted_X = sorted(X)
    n = len(X)
    x_values = sorted_X # Точка перед минимумом для нулевого
# значения
    cdf_values = [i / n for i in range(1, n + 1)] # Значения
# ЭФР
    x_values.append(max(sorted_X) + 0.005)
    cdf_values.append(1)
    return x_values, cdf_values
X = read_csv('sample.csv')
sizes = [10, 100, 200]
subsamples = {size: random.sample(X, min(size, len(X))) for size
in sizes}
# Построение отдельных графиков
for size, subsample in subsamples.items():
    x_vals, cdf_vals = cdf(subsample)
    plt.figure(figsize=(8, 6))
    plt.step(x_vals, cdf_vals, where='post', label=f'Размер
# подвыборки = {size}')

    plt.title(f'Эмпирическая функция распределения (n =
# {size})', fontsize=14)
    plt.xlabel('x', fontsize=12)
    plt.ylabel('F_n(x)', fontsize=12)
    plt.legend()
    plt.grid(True)
    plt.savefig(f'cdf_n{size}.png')
    plt.close()
```

ПРИЛОЖЕНИЕ 3

Листинг кода, для построения графика теоретической функции распределения Гомперца при различных параметрах для сравнения с эмпирической функцией распределения:

```
import matplotlib.pyplot as plt
import math

# Теоретическая функция распределения Гомперца
def gompertz_cdf(x, c, loc):
    if x < loc:
        return 0
    return 1 - math.exp(-(c) * (math.exp((x - loc)) - 1))

# Диапазон значений x
x_values = [x * 0.01 for x in range(600, 801)]
# Список параметров для построения
parameters = [
    (0.1, 6.0, 'c = 0.1, loc = 6.0'),
    (0.5, 6.2, 'c = 0.5, loc = 6.2'),
    (1.0, 6.4, 'c = 1.0, loc = 6.4'),
    (5.0, 6.6, 'c = 5.0, loc = 6.6'),
    (10.0, 6.8, 'c = 10.0, loc = 6.8'),
]
# Построение графика
plt.figure(figsize=(10, 6))
for c, loc, label in parameters:
    cdf_values = [gompertz_cdf(x, c, loc) for x in x_values]
    plt.plot(x_values, cdf_values, label=label)
plt.title('Теоретическая функция распределения Гомперца',
          fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('F(x)', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('gompertz_cdf.png')
plt.show()
```

ПРИЛОЖЕНИЕ 4

Листинг кода, предназначенного для построения гистограмм и теоретической функции плотности распределения Гомперца:

```
import csv
import random
import matplotlib.pyplot as plt
import math

# Чтение выборки из файла .csv
def read_csv_column(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X

# Функция для построения гистограммы
def build_histogram(X, num_bins):
    x_min, x_max = min(X), max(X)
    bin_width = (x_max - x_min) / num_bins
    bins = [x_min + i * bin_width for i in range(num_bins + 1)]

    frequencies = [0] * num_bins
    for x in X:
        for i in range(num_bins):
            if bins[i] <= x < bins[i + 1]:
                frequencies[i] += 1
                break
        if x == x_max:
            frequencies[-1] += 1

    n = len(X)
    densities = [freq / (n * bin_width) for freq in frequencies]
    return bins, densities

# Теоретическая плотность Гомперца
def gompertz_pdf(x, c, loc):
    return c * math.exp(x - loc - c * (math.exp(x - loc) - 1))

# Чтение выборки
X = read_csv_column('sample.csv')

# Создание подвыборок
sizes = [10, 100, 200]
subsamples = {size: random.sample(X, min(size, len(X))) for size
```

```

in sizes}

# Построение гистограмм
for size, subsample in subsamples.items():
    num_bins = int(size ** 0.5)
    bins, densities = build_histogram(subsample, num_bins)
    plt.figure(figsize=(8, 6))
    plt.bar(bins[:-1], densities, width=[bins[i + 1] - bins[i]
for i in range(len(bins) - 1)], align='edge',
            color='blue', edgecolor='black', alpha=0.7,
label=f'n = {size}')
    plt.title(f'Гистограмма выборки (n = {size})', fontsize=14)
    plt.xlabel('x', fontsize=12)
    plt.ylabel('Плотность', fontsize=12)
    plt.legend()
    plt.grid(True)
    plt.savefig(f'histogram_n{size}.png')
    plt.close()

x_theoretical = [i * 0.01 for i in range(635, 700)]
parameters = [
    (1.0, 6.350, 'c = 1.0, loc = 6.350'),
    (5.0, 6.350, 'c = 5.0, loc = 6.350'),
    (8.0, 6.351, 'c = 8.0, loc = 6.351'),
    (10.0, 6.351, 'c = 10.0, loc = 6.351'),
    (15.0, 6.352, 'c = 15.0, loc = 6.352'),
]

plt.figure(figsize=(8, 6))
for c, loc, label in parameters:
    pdf_values = [gompertz_pdf(x, c, loc) for x in
x_theoretical]
    plt.plot(x_theoretical, pdf_values, label=label,
linestyle='--')

plt.title('Теоретическая плотность Гомперца', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('Плотность', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('gompertz_pdf.png')
plt.show()

```

ПРИЛОЖЕНИЕ 5

Листинг кода для построения теоретической функции распределения с параметрами $c = 0.05$, $\text{loc} = 6.0$:

```
import matplotlib.pyplot as plt
import math

# Теоретическая функция распределения Гомперца
def gompertz_cdf(x, c, loc):
    if x < 0:
        return 0
    return 1 - math.exp(-(c) * (math.exp(x - loc) - 1))

x_values = [i * 0.01 for i in range(550, 1500)] # x от 0 до 10

plt.figure(figsize=(8, 6))
loc = 6
c = 0.05

cdf_values = [gompertz_cdf(x, c, loc) for x in x_values]
plt.plot(x_values, cdf_values, label=f'c = {c}, loc = {loc}')

plt.title('Теоретическая функция распределения Гомперца F(x; c, loc)', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('F(x)', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('gompertz_cdf_random.png')
plt.close()
```

ПРИЛОЖЕНИЕ 6

Листинг кода, предназначенного для оценки влияния параметров c и loc на график функции распределения:

```
import matplotlib.pyplot as plt
import math

# Теоретическая функция распределения Гомперца
def gompertz_cdf(x, c, loc):
    if x < 0:
        return 0
    return 1 - math.exp(-(loc / c) * (math.exp(c * x) - 1))

x_values = [i * 0.01 for i in range(0, 1001)] # x от 0 до 10

# Влияние параметра c
plt.figure(figsize=(8, 6))
# Фиксируем loc
loc_fixed = 0.5
# Два значения для c
c_values = [0.1, 1.0]
for c in c_values:
    cdf_values = [gompertz_cdf(x, c, loc_fixed) for x in
x_values]
    plt.plot(x_values, cdf_values, label=f'c = {c}, loc =
{loc_fixed}')

plt.title('Влияние параметра c на функцию распределения
Гомперца', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('F(x)', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('gompertz_cdf_c_variation.png')
plt.close()

# Влияние параметра loc
plt.figure(figsize=(8, 6))
# Фиксируем c
c_fixed = 0.2
# Два значения для loc
loc_values = [0.1, 1.0]
for loc in loc_values:
    cdf_values = [gompertz_cdf(x, c_fixed, loc) for x in
x_values]
    plt.plot(x_values, cdf_values, label=f'c = {c_fixed}, loc =
{loc}')

plt.title('Влияние параметра loc на функцию распределения
Гомперца', fontsize=14)
plt.xlabel('x', fontsize=12)
```

```
plt.ylabel('F(x)', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('gompertz_cdf_loc_variation.png')
plt.close()
```

ПРИЛОЖЕНИЕ 7

Листинг кода, для вычисления точечных оценок параметров c и loc методом максимального правдоподобия:

```
import math
import csv

# Чтение выборки из файла sample.csv
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X

# Вычисление оценок методом максимального правдоподобия
def mle_gompertz(X):
    n = len(X)
    loc_hat = min(X)
    sum_exp = sum(math.exp(x - loc_hat) - 1 for x in X)
    if sum_exp <= 0:
        raise ValueError("Сумма (e^(x - loc) - 1) должна быть положительной")
    c_hat = n / sum_exp
    return c_hat, loc_hat

X = read_csv('sample.csv')
c_hat, loc_hat = mle_gompertz(X)
print(f"Оценка параметра c (форма): {c_hat:.6f}")
print(f"Оценка параметра loc (смещение): {loc_hat:.6f}")
```


ПРИЛОЖЕНИЕ 8

Листинг кода, для вычисления точечных оценок параметров c и $\log$

МЕТОДОМ МОМЕНТОВ:

```
import numpy as np
from scipy.integrate import quad
from scipy.optimize import fsolve
import csv

# Чтение выборки из файла sample.csv
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X

# Подынтегральная функция для  $E[U]$ 
def integrand_mu(u, c):
    exponent = u - c * (np.exp(u) - 1)
    return np.where(exponent < -700, 0, u * c *
np.exp(exponent))

# Подынтегральная функция для  $M[U^2]$ 
def integrand_mu2(u, c):
    exponent = u - c * (np.exp(u) - 1)
    return np.where(exponent < -700, 0, u ** 2 * c *
np.exp(exponent))

# Вычисление  $M[U]$  и  $M[U^2]$  численно
def compute_moments_numerically(c):
    u_max = 50
    mu, _ = quad(integrand_mu, 0, u_max, args=(c,))
    mu2, _ = quad(integrand_mu2, 0, u_max, args=(c,))
    return mu, mu2

# Функция для уравнения дисперсии
def variance_equation(c, target_variance):
    c = float(c[0]) if isinstance(c, (list, np.ndarray)) else c
    mu, mu2 = compute_moments_numerically(c)
    return (mu2 - mu ** 2) - target_variance

# Вычисление оценок методом моментов
def moments_gompertz_numerical(sample):
    n = len(sample)
    # Выборочное среднее
    mean = sum(sample) / n
    # Выборочная дисперсия
    mean_square = sum(x ** 2 for x in sample) / n
```

```

variance = mean_square - mean ** 2
# Численное решение для c
c_initial = 0.1 # Начальное приближение
c_hat = fsolve(variance_equation, c_initial,
args=(variance,))[0]
# Вычисляем M[U] для найденного c
mu, _ = compute_moments_numerically(c_hat)
# Оценка loc
loc_hat = mean - mu
return c_hat, loc_hat

sample = read_csv('sample.csv')
c_hat, loc_hat = moments_gompertz_numerical(sample)
print(f"Оценка параметра c (форма): {c_hat:.6f}")
print(f"Оценка параметра loc (смещение): {loc_hat:.6f}")

```

ПРИЛОЖЕНИЕ 9

Листинг кода для построения графиков функции распределения – теоретических с вычисленными значениями параметров и эмпирической:

```
import numpy as np
import matplotlib.pyplot as plt
import csv

# Чтение выборки из файла sample.csv
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return X

# Теоретическая функция распределения Гомперца
def gompertz_cdf(x, c, loc):
    if x < loc:
        return 0
    return 1 - np.exp(-c * (np.exp(x - loc) - 1))

# Эмпирическая функция распределения
def empirical_cdf(sample):
    sorted_sample = sorted(sample)
    n = len(sample)
    x_values = sorted_sample
    cdf_values = [i / n for i in range(1, n + 1)]
    x_values = [min(sorted_sample) - 0.01] + x_values
    cdf_values = [0] + cdf_values
    return x_values, cdf_values

sample = read_csv('sample.csv')
# Параметры, полученные методом моментов
c_mm = 10.221538
loc_mm = 6.358577
# Параметры, полученные методом максимального правдоподобия
c_ml = 9.453330
loc_ml = 6.351309
# Диапазон x для построения теоретических функций
x_min = min(sample) - 0.01
x_max = max(sample) + 0.1
x_values = np.linspace(x_min, x_max, 100)
# Вычисляем значения теоретических функций распределения
cdf_mm = [gompertz_cdf(x, c_mm, loc_mm) for x in x_values]
cdf_ml = [gompertz_cdf(x, c_ml, loc_ml) for x in x_values]
# Вычисляем эмпирическую функцию распределения
x_emp, cdf_emp = empirical_cdf(sample)
plt.figure(figsize=(10, 6))
```

```

plt.plot(x_values, cdf_mm, label=f'Метод моментов (c={c_mm:.2f},
loc={loc_mm:.2f})', color='blue')
plt.plot(x_values, cdf_ml, label=f'Метод ММП (c={c_ml:.2f},
loc={loc_ml:.2f})', color='red')
plt.step(x_emp, cdf_emp, label='Эмпирическая функция
распределения', color='green', where='post')
plt.title('Сравнение теоретических и эмпирической функций
распределения', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('F(x)', fontsize=12)
plt.legend()
plt.grid(True)
plt.savefig('comparison_cdf.png')

```

ПРИЛОЖЕНИЕ 10

Листинг программы для проверки эффективности оценок:

```
import numpy as np
from scipy.integrate import quad
from scipy.optimize import fsolve
import math

# Генерация выборки из Гомперца
def generate_gompertz_sample(n, c, loc):
    u = np.random.uniform(0, 1, n)
    x = loc + np.log(1 - np.log(1 - u) / c)
    return x

# Реализация метода моментов (аналогично исходному коду)
def integrand_mu(u, c):
    exponent = u - c * (np.exp(u) - 1)
    return np.where(exponent < -700, 0, u * c *
np.exp(exponent))

def integrand_mu2(u, c):
    exponent = u - c * (np.exp(u) - 1)
    return np.where(exponent < -700, 0, u ** 2 * c *
np.exp(exponent))

def compute_moments_numerically(c):
    u_max = 50
    mu, _ = quad(integrand_mu, 0, u_max, args=(c,))
    mu2, _ = quad(integrand_mu2, 0, u_max, args=(c,))
    return mu, mu2

def variance_equation(c, target_variance):
    c = float(c[0]) if isinstance(c, (list, np.ndarray)) else c
    mu, mu2 = compute_moments_numerically(c)
    return (mu2 - mu ** 2) - target_variance

def moments_gompertz_numerical(sample):
    n = len(sample)
    mean = sum(sample) / n
    mean_square = sum(x ** 2 for x in sample) / n
    variance = mean_square - mean ** 2
    c_initial = 0.1
    c_hat = fsolve(variance_equation, c_initial,
args=(variance,))[0]
    mu, _ = compute_moments_numerically(c_hat)
    loc_hat = mean - mu
    return c_hat, loc_hat
```

```

# Реализация метода ММП (аналогично исходному коду)
def mle_gompertz(X):
    n = len(X)
    loc_hat = min(X)
    sum_exp = sum(math.exp(x - loc_hat) - 1 for x in X)
    if sum_exp <= 0:
        raise ValueError("Сумма (e^(x - loc) - 1) должна быть
положительной")
    c_hat = n / sum_exp
    return c_hat, loc_hat

# Симуляция
n = 300
true_c = 9.453330
true_loc = 6.351309
num_simulations = 1000

moments_c_estimates = []
moments_loc_estimates = []
mle_c_estimates = []
mle_loc_estimates = []

for _ in range(num_simulations):
    sample = generate_gompertz_sample(n, true_c, true_loc)
    c_mom, loc_mom = moments_gompertz_numerical(sample)
    moments_c_estimates.append(c_mom)
    moments_loc_estimates.append(loc_mom)
    c_mle, loc_mle = mle_gompertz(sample)
    mle_c_estimates.append(c_mle)
    mle_loc_estimates.append(loc_mle)

moments_c_mean = np.mean(moments_c_estimates)
moments_loc_mean = np.mean(moments_loc_estimates)
mle_c_mean = np.mean(mle_c_estimates)
mle_loc_mean = np.mean(mle_loc_estimates)

moments_c_bias = moments_c_mean - true_c
moments_loc_bias = moments_loc_mean - true_loc
mle_c_bias = mle_c_mean - true_c
mle_loc_bias = mle_loc_mean - true_loc

moments_c_var = np.var(moments_c_estimates, ddof=1)
moments_loc_var = np.var(moments_loc_estimates, ddof=1)
mle_c_var = np.var(mle_c_estimates, ddof=1)
mle_loc_var = np.var(mle_loc_estimates, ddof=1)

# Информация Фишера для c и loc
I_c = 1 / (true_c ** 2) # I(c) = 1/c^2
I_loc = true_c * (1 + 1 / true_c) # I(loc) = c (1 + 1/c)

```

```

# CRLB
crlb_c_mm = (1 + moments_c_bias) ** 2 / (n * I_c)
crlb_loc_mm = 1 / (n * true_c) ** 2
crlb_c_mle = (1 + mle_c_bias) ** 2 / (n * I_c)
crlb_loc_mle = 1 / (n * true_c) ** 2

# Вывод результатов
print("Метод моментов:")
print(f"Смещение для c: {moments_c_bias:.6f}")
print(f"Смещение для loc: {moments_loc_bias:.6f}")
print(f"Дисперсия для c: {moments_c_var:.6e}")
print(f"Дисперсия для loc: {moments_loc_var:.6e}")
print("Нижняя граница Рао-Крамера:")
print(f"для c: {crlb_c_mm:.6e}")
print(f"для loc: {crlb_loc_mm:.6e}")

print("\nМетод ММП:")
print(f"Смещение для c: {mle_c_bias:.6f}")
print(f"Смещение для loc: {mle_loc_bias:.6f}")
print(f"Дисперсия для c: {mle_c_var:.6e}")
print(f"Дисперсия для loc: {mle_loc_var:.6e}")
print("Нижняя граница Рао-Крамера:")
print(f"для c: {crlb_c_mle:.6e}")
print(f"для loc: {crlb_loc_mle:.6e}")

```

ПРИЛОЖЕНИЕ 11

Листинг кода для вычисления доверительного интервала для параметра loc :

```
import numpy as np
from scipy.integrate import quad
from scipy.optimize import root_scalar
n = 300
gamma_1 = 0.05
gamma_2 = 0.01
gamma_1_n = gamma_1**(1/n)
gamma_2_n = gamma_2**(1/n)
c_true, loc_true = 9.453330, 6.351309

# Функция плотности  $f(u; c, 0)$ 
def gompertz_density(u, c):
    return c * np.exp(u - c * (np.exp(u) - 1))

# Функция для интегрирования
def integral_from_z(z, c):
    result, _ = quad(gompertz_density, z, 10, args=(c,))
    return result

# Функция для решения уравнения
def equation(z, c, target):
    return integral_from_z(z, c) - target

# Решение для  $z$ 
z_95 = root_scalar(equation, args=(c_true, gamma_1_n),
    bracket=[0, 10]).root
z_99 = root_scalar(equation, args=(c_true, gamma_2_n),
    bracket=[0, 10]).root
# Доверительные интервалы
ci_loc_95 = [loc_true - z_95, loc_true]
ci_loc_99 = [loc_true - z_99, loc_true]
print(f"γ=0.95: loc ∈ {ci_loc_95}")
print(f"γ=0.99: loc ∈ {ci_loc_99}")
```


ПРИЛОЖЕНИЕ 12

Листинг кода для критерия отношения правдоподобия:

```
import numpy as np
from scipy.optimize import minimize
from scipy.stats import chi2
import math
import csv

# Чтение выборки из файла
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return np.array(X)

# ММП-оценки для распределения Гомперца
def mle_gompertz(X):
    n = len(X)
    loc_hat = min(X)
    sum_exp = sum(math.exp(x - loc_hat) - 1 for x in X)
    c_hat = n / sum_exp
    return c_hat, loc_hat

# Логарифм правдоподобия
def log_likelihood(c, loc, sample, loc_base):
    filtered_sample = sample[sample >= loc_base]
    n = len(filtered_sample)
    if n == 0 or c <= 0 or loc < loc_base:
        return -np.inf
    term1 = n * np.log(c)
    term2 = np.sum(filtered_sample - loc)
    term3 = -c * np.sum(np.exp(filtered_sample - loc) - 1)
    return term1 + term2 + term3

def likelihood(c, loc, sample, loc_base):
    log_L = log_likelihood(c, loc, sample, loc_base)
    if log_L == -np.inf:
        return 0, log_L
    L = np.exp(log_L)
    return L, log_L

# LR-критерий
def likelihood_ratio_test(sample, c_mle_initial,
                           loc_mle_initial, c_hat, loc_hat, method_name, loc_base):
    n = len(sample)
    print(f"\nАнализ для метода {method_name}:")

    filtered_sample = sample[sample >= loc_base]
    c_mle, loc_mle = mle_gompertz(filtered_sample)
```

```

    print(f"Пересчитанные ММП-оценки: c_mle = {c_mle:.6f},
loc_mle = {loc_mle:.6f}")
    L1_star, log_L1_star = likelihood(c_mle, loc_mle, sample,
loc_base)
    print(f"L* = {L1_star:.6e}, ln(L*) = {log_L1_star:.6f}")

    # 1. Проверка H_0: c = c_hat
    print(f"\nПроверка H_0: c = {c_hat:.6f}")
    # Максимизация по loc при фиксированном c
    result_loc = minimize(lambda loc: -log_likelihood(c_hat,
loc, sample, loc_base),
loc_mle, bounds=[(loc_base,
np.max(sample) + 0.1)])
    loc_opt = result_loc.x[0]
    L_temp, log_L_temp = likelihood(c_hat, loc_opt, sample,
loc_base)
    print(f"Оптимальное loc при c = {c_hat:.6f}: {loc_opt:.6f}")
    L1_prime, log_L1_prime = likelihood(c_hat, loc_opt, sample,
loc_base)
    print(f"L0* = {L1_prime:.6e}, ln(L0*) = {log_L1_prime:.6f}")

    ratio1 = np.exp(abs(log_L1_star - log_L1_prime))
    lambda_c = 2 * (abs(log_L1_star - log_L1_prime))
    print(f"Отношение правдоподобия L*/L0* = {ratio1:.6f}")
    print(f"2*ln(L*/L0*) = {lambda_c:.6f}")

    # 2. Проверка H_0: loc = loc_hat
    print(f"\nПроверка H_0: loc = {loc_hat:.6f}")
    # Максимизация по c при фиксированном loc
    result_c = minimize(lambda c: -log_likelihood(c, loc_hat,
sample, loc_base), c_mle, bounds=[(0.1, None)])
    c_opt = result_c.x[0]
    L1_double_prime, log_L1_double_prime = likelihood(c_opt,
loc_hat, sample, loc_base)
    print(f"Оптимальное c при loc = {loc_hat:.6f}: {c_opt:.6f}")
    print(f"L0* = {L1_double_prime:.6e}, ln(L0*) =
{log_L1_double_prime:.6f}")
    ratio2 = np.exp(log_L1_star - log_L1_double_prime)
    lambda_loc = 2 * (log_L1_star - log_L1_double_prime)
    print(f"Отношение правдоподобия L*/L0* = {ratio2:.6f}")
    print(f"2*ln(L*/L0*) = {lambda_loc:.6f}")

    # Критическое значение
    alpha = 0.05
    C = chi2.ppf(1 - alpha, df=1)
    print(f"\n(1-alpha)-квантиль  $\chi^2(1)$  при alpha = {alpha}:
{C:.6f}")

    # Вывод
    print("\nВывод:")
    print(f"Для H_0: c = {c_hat:.6f}: lambda = {lambda_c:.6f}, C
= {C:.6f}")
    print("Отклоняем H_0" if lambda_c > C else "Принимаем H_0")

```

```

        print(f"Для H_0: loc = {loc_hat:.6f}: lambda =
{lambda_loc:.6f}, C = {C:.6f}")
        print("Отклоняем H_0" if lambda_loc > C else "Принимаем
H_0")
        return lambda_c, lambda_loc, C

n = 300
true_c = 9.453330
true_loc = 6.351309
sample = read_csv("sample.csv")

# Начальные оценки методом ММП
c_mle_initial = 9.453330
loc_mle_initial = 6.351309

# Оценки методом моментов
c_mom = 10.221538
loc_mom = 6.358577

lambda_mom_c, lambda_mom_loc, C_mom =
likelihood_ratio_test(sample, c_mle_initial, loc_mle_initial,
c_mom, loc_mom, "моментов", loc_mom)

```

ПРИЛОЖЕНИЕ 13

Листинг кода для проверки гипотез о виде распределения:

```
import numpy as np
from scipy.stats import chi2, kstest
import csv
import math

# Чтение выборки из файла
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return np.array(X)

# Теоретическая функция распределения Гомперца
def gompertz_cdf(x, c=9.453330, loc=6.351309):
    if x < loc:
        return 0.0
    return 1 - np.exp(-c * (np.exp(x - loc) - 1))

# Критерий Колмогорова
def kolmogorov_test(sample):
    n = len(sample)
    sample_sorted = np.sort(sample)
    empirical_cdf = np.arange(1, n + 1) / n
    theoretical_cdf = np.array([gompertz_cdf(x) for x in
sample_sorted])
    differences = np.abs(empirical_cdf - theoretical_cdf)
    D = np.max(differences)
    # Индекс максимальной разницы
    max_index = np.argmax(differences)
    max_x = sample_sorted[max_index]
    max_Fn = empirical_cdf[max_index]
    max_F = theoretical_cdf[max_index]
    max_diff = differences[max_index]

    step = 10
    indices_to_show = list(range(0, n, step))
    if max_index not in indices_to_show:
        indices_to_show.append(max_index)
        indices_to_show.sort()

    print("\nКритерий Колмогорова:")
    print("\nПромежуточные вычисления (каждый 10-й элемент):")
    print(f"{'x_i':<15} {'F_n(x_i)':<15} {'F(x_i)':<15} {'|F_n -
F|':<15}")
```

```

print("-" * 60)
for i in indices_to_show:
    print(
        f"{sample_sorted[i]:<15.6f}
{empirical_cdf[i]:<15.6f} {theoretical_cdf[i]:<15.6f}
{differences[i]:<15.6f}")
    print("-" * 60)
    print(f"Максимальная разница  $|F_n - F| = \{max\_diff:.6f\}$ 
достигнута при  $x = \{max\_x:.6f\}$ ")
    print(f" $F_n(x) = \{max\_Fn:.6f\}$ ,  $F(x) = \{max\_F:.6f\}$ ")
    D_critical = 1.36 / np.sqrt(n)
    print(f"Критическое значение  $D_{критический} =$ 
{D_critical:.6f}")
    return D < D_critical

# Критерий  $\chi^2$ 
def chi2_test(sample, k=9):
    n = len(sample)
    c, loc = 9.453330, 6.351309
    min_x, max_x = np.min(sample), np.max(sample)
    bins = np.linspace(min_x, max_x, k + 1)
    observed_freq, _ = np.histogram(sample, bins)
    expected_freq = np.zeros(k)
    probabilities = np.zeros(k)
    print("\nКритерий Хи-квадрат:")
    print(f"{'Интервал':<20} {'Вероятность p_i':<20} {'Ожидаемая
частота n*p_i':<25} {'Наблюдаемая частота':<20}")
    print("-" * 90)
    for i in range(k):
        p1 = gompertz_cdf(bins[i + 1], c, loc)
        p0 = gompertz_cdf(bins[i], c, loc)
        probabilities[i] = p1 - p0
        expected_freq[i] = n * probabilities[i]
        print(
            f"[{bins[i]:.4f}, {bins[i + 1]:.4f}] {'':<10}
{probabilities[i]:.6f} {'':<10} {expected_freq[i]:.6f} {'':<10}
{observed_freq[i]}")
        chi2_stat = np.sum((observed_freq - expected_freq) ** 2 /
expected_freq)
        df = k - 1 - 2
        chi2_critical = chi2.ppf(1 - 0.05, df)
        print(f"\nКритерий  $\chi^2$ :")
        print(f"Степени свободы: {df}")
        print(f"Статистика  $\chi^2_{набл} = \{chi2\_stat:.6f\}$ ")
        print(f"Критическое значение  $\chi^2_{кр} = \{chi2\_critical:.6f\}$ ")
        return chi2_stat < chi2_critical

sample = read_csv("sample.csv")
n = len(sample)

kolmogorov_result = kolmogorov_test(sample)

```

```
print(f"Гипотеза принята (Колмогорова): {kolmogorov_result}")  
chi2_result = chi2_test(sample)  
print(f"Гипотеза принята ( $\chi^2$ ): {chi2_result}")
```

ПРИЛОЖЕНИЕ 14

Листинг кода для проверки гипотезы об однородности выборок:

```
import numpy as np
from scipy.stats import gamma
import csv
import matplotlib.pyplot as plt

# Чтение исходной выборки
def read_csv(filename):
    X = []
    with open(filename, 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            if row:
                X.append(float(row[0]))
    return np.array(X)

# Обратная функция для гаммы
def gamma_inv_shifted(u, k=1.36, theta=0.071, loc=6.351309):
    return gamma.ppf(u, a=k, scale=theta) + loc

# Генерация выборки X_2
def generate_sample_x2(size=300):
    u = np.random.uniform(0, 1, size)
    x2 = gamma_inv_shifted(u)
    return x2

# Критерий Колмогорова-Смирнова
def kolmogorov_smirnov_two_sample(x1, x2, alpha=0.05):
    n1, n2 = len(x1), len(x2)
    x1_sorted = np.sort(x1)
    x2_sorted = np.sort(x2)
    empirical_cdf1 = np.arange(1, n1 + 1) / n1
    empirical_cdf2 = np.arange(1, n2 + 1) / n2

    # Объединённая выборка для вычисления статистики
    combined = np.sort(np.concatenate([x1, x2]))
    cdf1_at_combined = np.interp(combined, x1_sorted,
    empirical_cdf1, left=0, right=1)
    cdf2_at_combined = np.interp(combined, x2_sorted,
    empirical_cdf2, left=0, right=1)
    D = np.max(np.abs(cdf1_at_combined - cdf2_at_combined))

    D_norm = D * np.sqrt((n1 * n2) / (n1 + n2))
    D_critical = 1.36
    print(f"\nКритерий Колмогорова-Смирнова:")
    print(f"Нормированная статистика D = {D_norm:.6f}")
    print(f"Критическое значение D_критический =
```

```

{D_critical:.6f}")

    result = D_norm < D_critical
    print(f"Гипотеза H_0 принята: {result}")
    return D_norm, D_critical, result

sample_x1 = read_csv("sample.csv")
n = len(sample_x1)
sample_x2 = generate_sample_x2(n)
D_norm, D_critical, result =
kolmogorov_smirnov_two_sample(sample_x1, sample_x2)

plt.step(np.sort(sample_x1), np.arange(1, n + 1) / n,
label="Эмпирическая функция выборки X1 (Гомперц)")
plt.step(np.sort(sample_x2), np.arange(1, n + 1) / n,
label="Эмпирическая функция выборки X2 (Гамма)")
plt.xlabel("x")
plt.ylabel("F^(x) ")
plt.legend()
plt.title("Сравнение эмпирических функций распределения")
plt.grid(True)
plt.savefig("edf_comparison.png")
plt.show()

```


ПРИЛОЖЕНИЕ 15

Листинг кода для оценки мощности критерия Колмогорова-Смирнова:

```
import numpy as np
from scipy.stats import gamma

# Гомперц обратная функция
def gompertz_inv(u, c=9.453330, loc=6.351309):
    return loc + np.log(1 - np.log(1 - u) / c)

# Генерация выборки Гомперца
def generate_gompertz_sample(size=300, c=9.453330,
loc=6.351309):
    u = np.random.uniform(0, 1, size)
    return gompertz_inv(u, c, loc)

# Генерация выборки гамма
def generate_gamma_sample(size=300, k=1.36, theta=0.071,
loc=6.351309):
    u = np.random.uniform(0, 1, size)
    return gamma.ppf(u, a=k, scale=theta) + loc

# Критерий Колмогорова-Смирнова
def kolmogorov_smirnov_two_sample(x1, x2):
    n1, n2 = len(x1), len(x2)
    x1_sorted = np.sort(x1)
    x2_sorted = np.sort(x2)
    empirical_cdf1 = np.arange(1, n1 + 1) / n1
    empirical_cdf2 = np.arange(1, n2 + 1) / n2

    combined = np.sort(np.concatenate([x1, x2]))
    cdf1_at_combined = np.interp(combined, x1_sorted,
empirical_cdf1, left=0, right=1)
    cdf2_at_combined = np.interp(combined, x2_sorted,
empirical_cdf2, left=0, right=1)
    D = np.max(np.abs(cdf1_at_combined - cdf2_at_combined))
    D_norm = D * np.sqrt((n1 * n2) / (n1 + n2))
    return D_norm

# Симуляция мощности
def estimate_power(n_simulations=1000):
    D_critical = 1.36
    rejections = 0
    for _ in range(n_simulations):
        x1 = generate_gompertz_sample()
        x2 = generate_gamma_sample()
        D_norm = kolmogorov_smirnov_two_sample(x1, x2)
        if D_norm > D_critical:
            rejections += 1
```

```
power = rejections / n_simulations
return power

power = estimate_power(n_simulations=1000)
print(f"Оцененная мощность критерия: {power:.4f}")
```